

**UNIVERSIDADE DE SÃO PAULO
ESCOLA DE ENGENHARIA DE SÃO CARLOS**

GUILHERME LIMA BLATT

**Desenvolvimento de um sistema de aquisição de dados e distribuição de energia em um
protótipo de Fórmula SAE**

São Carlos

2020

GUILHERME LIMA BLATT

Desenvolvimento de um sistema de aquisição de dados e distribuição de energia em um protótipo de Fórmula SAE

Monografia apresentada ao Curso de Engenharia de Computação, da Escola de Engenharia de São Carlos e Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo, como parte dos requisitos para obtenção do título de Engenheiro de Computação.

Orientador: Prof. Dr. Maximilian Luppe

São Carlos

2020

AUTORIZO A REPRODUÇÃO TOTAL OU PARCIAL DESTE TRABALHO, POR QUALQUER MEIO CONVENCIONAL OU ELETRÔNICO, PARA FINS DE ESTUDO E PESQUISA, DESDE QUE CITADA A FONTE.

Ficha catalográfica elaborada pela Biblioteca Prof. Dr. Sérgio Rodrigues Fontes da EESC/USP com os dados inseridos pelo(a) autor(a).

BB644d BLATT, Guilherme Lima
Desenvolvimento de um sistema de aquisição de dados e distribuição de energia em um protótipo de Fórmula SAE / Guilherme Lima Blatt; orientador Maximilian Luppe. São Carlos, 2020.

Monografia (Graduação em Engenharia de Computação) -- Escola de Engenharia de São Carlos e Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo, 2020.

1. Microcontrolador. 2. Aquisição de dados. 3. Telemetria. 4. Eletrônica embarcada. 5. Fórmula SAE. I. Título.

FOLHA DE APROVAÇÃO

Nome: Guilherme Lima Blatt

Título: “Desenvolvimento de um sistema de aquisição de dados e distribuição de energia em um protótipo de Fórmula SAE”

Trabalho de Conclusão de Curso defendido em 04/12/2020.

Comissão Julgadora:

Resultado:

Prof. Dr. Maximilian Luppe - Orientador -
SEL/EESC/USP

Aprovado

Profa. Associada Máira Martins da Silva
SEM/EESC/USP

Aprovado

Prof. Associado Leopoldo Pisanelli Rodrigues de
Oliveira
SEM/EESC/USP

Aprovado

**Coordenador do Curso Interunidades Engenharia de Computação pela
EESC/USP :**

Prof. Dr. Maximilian Luppe

*Dedico este trabalho a meu pai, minha mãe e
meu irmão, por me abrirem essa oportunidade.*

AGRADECIMENTOS

Agradeço meu pai, minha mãe e meu irmão, por sempre me apoiarem e possibilitarem essa oportunidade.

Agradeço a todos membros da equipe EESC USP Formula SAE, por compartilharem tantos momentos de dificuldades e conquistas, e contribuírem na minha formação como profissional.

Agradeço a meus amigos da Engenharia de Computação 016, por todos os momentos juntos e pela união. Sem ela, não chegaríamos aqui.

Agradeço à Universidade de São Paulo, aos professores e funcionários, que possibilitam o desenvolvimento e formação dos alunos.

Agradeço meu professor orientador, Maximilian Luppe, pelo apoio e incentivo desse trabalho.

RESUMO

BLATT, Guilherme Lima. **Desenvolvimento de um sistema de aquisição de dados e distribuição de energia em um protótipo de Fórmula SAE**. 2020. 96p. Monografia (Trabalho de Conclusão de Curso) – Escola de Engenharia de São Carlos, Universidade de São Paulo, São Paulo, 2020.

Em ambientes de competições automotivas, é de extrema importância entender como o carro se comporta para poder extrair seu maior desempenho, além de monitorar sinais importantes, como temperaturas e pressões, em tempo real, e garantir um sistema elétrico robusto que não falhe durante uma corrida. Assim, esse trabalho irá mostrar os passos no projeto e implementação do sistema elétrico e eletrônico desenvolvido para o protótipo de 2019 da equipe EESC USP Formula SAE, que incluem sistema de aquisição de dados, GPS, telemetria usando um XBee, gravador de dados em um cartão SD, atuadores pneumáticos para troca de marcha e um módulo de distribuição de energia utilizando MOSFETs para substituir relés mecânicos. Os processos descritos para obter tais resultados incluem análise de sensores e atuadores necessários, desenvolvimento das placas de circuito impresso, desenvolvimento do firmware embarcado dos microcontroladores, interface com sensores e atuadores, o software de recebimento dos dados pela telemetria, também responsável por outras funções como salvamento dos dados, entre outros. Porém, de pouco adianta desenvolver tal sistema se não for possível utilizar os dados obtidos da maneira correta, assim também serão demonstrados exemplos de como utilizar os dados para melhorar o desempenho do veículo, aperfeiçoar o estilo de pilotagem, e ferramentas de automatização para permitir que seja possível processar uma quantidade grande de dados de maneira rápida e efetiva.

Palavras-chave: Microcontrolador. Aquisição de dados. Telemetria. Eletrônica Embarcada. Fórmula SAE.

ABSTRACT

BLATT, Guilherme Lima. **Development of a data acquisition and energy distribution system for a Formula SAE prototype**. 2020. 96p. Monografia (Trabalho de Conclusão de Curso) – Escola de Engenharia de São Carlos, Universidade de São Paulo, São Paulo, 2020.

In automotive competition environments, it is extremely important to understand the car's behavior to be able to extract its maximum performance, as well as monitor vital signals, such as temperatures and pressures, in real time and guarantee a robust electrical system that does not fail during a race. With that in mind, this thesis will show the steps related to the project and implementation of an electric and electronic system developed for the 2019 prototype of the EESC USP Formula SAE team, that includes a data acquisition, GPS, telemetry using XBee, datalogger using an SD card, pneumatic actuators for changing gears and a power distribution module that utilizes MOSFETs to replace mechanical relays. The steps described to achieve such results include analyzing which sensors and actuators are needed, development of printed circuit boards, writing the microcontroller firmware, interfacing the sensors and actuators and using the software that receives data from the telemetry. However, it is of little use to develop such system without using the obtained data correctly, so there will also be shown examples on how to use such data to make the vehicle performance better, perfecting driving styles, and automation tools to allow for a large amount of data to be processed quickly.

Keywords: Microcontroller. Data acquisition. Telemetry. Embedded electronics. Formula SAE.

LISTA DE ILUSTRAÇÕES

Figura 1: Protótipo da equipe EESC-USP Fórmula SAE	27
Figura 2: Estrutura transistor MOS	29
Figura 3: Regiões de operação transistor MOS	30
Figura 4: Aquisição de um sinal senoidal a várias frequências diferentes	31
Figura 5: Amplitude do diagrama de bode para filtro passa-baixa	32
Figura 6: Exemplo barramento CAN	33
Figura 7: Níveis de tensão no barramento CAN	34
Figura 8: Estrutura de uma mensagem CAN	34
Figura 9: Esquemático opto-isolador	35
Figura 10: Circuito dos reguladores de tensão	38
Figura 11: Implementação do MCP2561	39
Figura 12: Circuito oscilador	40
Figura 13: Circuito do programador	41
Figura 14: Capacitores de <i>decouple</i>	41
Figura 15: Implementação do microcontrolador	42
Figura 16: Utilização de LED	43
Figura 17: Conector automotivo	43
Figura 18: Filtro passa-baixa	44
Figura 19: Amplitude do diagrama de bode filtro passa-baixa	45
Figura 20: Circuito do adaptador cartão SD	45
Figura 21: Adaptador cartão SD	46
Figura 22: Entrada digital com <i>pullup</i>	46
Figura 23: Entrada digital com opto-isolador	47
Figura 24: Saída digital com opto-isolador	48
Figura 25: Circuito transmissor <i>XBee Pro 900HP</i>	48
Figura 26: <i>XBee Pro 900HP</i>	49
Figura 27: Circuito GPS	49
Figura 28: GPS <i>u-blox NEO-6M</i>	50
Figura 29: Circuito de uma saída de potência	51
Figura 30: Configuração <i>clock</i>	52
Figura 31: Configuração da porta	57
Figura 32: Configuração da porta serial	60

Figura 33: PCI do acelerômetro e giroscópio	64
Figura 34: Sensor de pressão de freio	65
Figura 35: Posicionamento sensor de esterço	66
Figura 36: Sensor de curso da suspensão	67
Figura 37: Posicionamento sensor de curso da suspensão no carro	67
Figura 38: Traçado do carro gerado pelo GPS	68
Figura 39: Sistema manual de troca de marchas	69
Figura 40: Sistema pneumático	70
Figura 41: Volante do protótipo	71
Figura 42: Componentes asa traseira	74
Figura 43: Asa traseira com <i>flaps</i> fechados	75
Figura 44: Asa traseira com flaps abertos	75
Figura 45: Programa utilizado para mostrar dados da telemetria	78
Figura 46: Sistema de detecção de falhas	78
Figura 47: Programa usado para analisar dados	79
Figura 48: Comparação entre pilotos	81
Figura 49: Sobreposição de dados em vídeo	83
Figura 50: Histograma posição do acelerador	84
Figura 51: Distribuição de freio por pressão de freio dianteira	85
Figura 52: Gráfico de dispersão do ângulo de rolagem por aceleração lateral	86
Figura 53: Progressão do tempo de volta em um teste	87
Figura 54: Temperatura média de óleo em um teste	88
Figura 55: Comparação tempo de volta entre pilotos	89
Figura 56: Comparação do acelerador entre pilotos	90
Figura 57: Contra esterço ao longo das voltas	91
Figura 58: Contra esterço por tempo de volta	92

LISTA DE TABELAS

Tabela 1: Pontuação da competição	28
Tabela 2: Parte dos dados enviados na rede CAN	55

LISTA DE CÓDIGOS

Código 1: Incrementação dos contadores	53
Código 2: Configuração da rede CAN	54
Código 3: Recebimento de mensagens CAN	56
Código 4: Envio de mensagem CAN	56
Código 5: Interrupção externa	58
Código 6: Envio de dados na porta serial	61
Código 7: Inicialização do programa	62
Código 8: Estrutura de repetição principal	63
Código 9: Interrupção do botão da embreagem	71
Código 10: Acionamento do atuador da embreagem	71
Código 11: Interrupção das trocas de marcha	72
Código 12: Acionamento do atuador de troca de marchas	73
Código 13: Controle do acionamento DRS	76

LISTA DE ABREVIATURAS E SIGLAS

ADC	<i>Analog-to-Digital Converter</i>
API	<i>Application Programming Interface</i>
ARB	<i>Anti-Roll Bar</i>
Bps	<i>Bytes per second</i>
bps	<i>bits per second</i>
CAN	<i>Controller Area Network</i>
CI	<i>Circuito Integrado</i>
DMA	<i>Direct Memory Access</i>
DRS	<i>Drag Reduction System</i>
ECU	<i>Engine Control Unit</i>
EESC	<i>Escola de Engenharia de São Carlos</i>
GPIO	<i>General Purpose Input/Output</i>
GPS	<i>Global Positioning System</i>
I2C	<i>Inter-Integrated Circuit</i>
IO	<i>Input/Output</i>
LED	<i>Light Emitting Diode</i>
PCB	<i>Printed Circuit Board</i>
PCI	<i>Placa de Circuito Impresso</i>
PWM	<i>Pulse Width Modulation</i>
SAE	<i>Society of Automotive Engineers</i>
SD	<i>Secure Digital</i>
SPI	<i>Serial Peripheral Interface</i>
SRAM	<i>Static Random Access Memory</i>
UART	<i>Universal Asynchronous Receiver/Transmitter</i>
USB	<i>Universal Serial Bus</i>
USP	<i>Universidade de São Paulo</i>
VBA	<i>Visual Basic for Applications</i>

SUMÁRIO

1	INTRODUÇÃO	23
1.1	APRESENTAÇÃO	23
1.2	MOTIVAÇÃO E OBJETIVOS	23
1.3	ESTRUTURA DESSE DOCUMENTO	25
2	EMBASAMENTO TEÓRICO	27
2.1	FÓRMULA SAE	27
2.2	TRANSISTORES MOS	28
2.3	AQUISIÇÃO DE SINAIS	31
2.4	BARRAMENTO CAN	32
2.5	OPTO-ISOLADOR	35
3	DESENVOLVIMENTO	37
3.1	DESENVOLVIMENTO DE HARDWARE	37
3.1.1	<i>Circuitos em comum</i>	37
3.1.1.1	Regulador de tensão	38
3.1.1.2	Interface CAN	39
3.1.1.3	Microcontrolador	39
3.1.1.4	Conector	43
3.1.2	<i>Módulo de Aquisição</i>	44
3.1.3	<i>Módulo de Controle</i>	46
3.1.4	<i>Módulo Transmissor</i>	48
3.1.5	<i>Módulo de Distribuição de Energia</i>	50
3.2	FIRMWARE	51
3.2.1	<i>Clock</i>	52
3.2.2	<i>Timer</i>	52
3.2.3	<i>Rede CAN</i>	54
3.2.4	<i>Entradas e Saídas</i>	57
3.2.5	<i>Entradas Analógicas</i>	59
3.2.6	<i>Comunicação Serial</i>	59
3.2.7	<i>Estrutura de execução</i>	61
3.3	SENSORES E ATUADORES	64

3.3.1	<i>Sensores analógicos</i>	64
3.3.1.1	<i>Acelerômetro e Giroscópio</i>	64
3.3.1.2	<i>Pressão de Freio</i>	65
3.3.1.3	<i>Esterço</i>	65
3.3.1.4	<i>Curso de suspensão</i>	66
3.3.2	<i>Sistema de Posicionamento Global (GPS)</i>	68
3.3.3	<i>Atuadores Pneumáticos</i>	69
3.3.4	<i>Sistema de Redução de Arrasto</i>	74
4	ANÁLISE DE DADOS	77
4.1	TELEMETRIA	77
4.1.1	<i>Transmissão dos dados</i>	77
4.1.2	<i>Recebimento dos dados</i>	77
4.2	DADOS ARMAZENADOS NO CARRO	79
4.2.1	<i>Análise de pilotagem</i>	80
4.2.2	<i>Análise do comportamento do veículo</i>	83
4.2.3	<i>Indicadores de desempenho</i>	87
5	CONCLUSÃO	93
5.1	TRABALHOS FUTUROS	94
	REFERÊNCIAS	95

1 INTRODUÇÃO

Nesse capítulo encontram-se a apresentação do projeto, as motivações e objetivos de sua realização.

1.1 Apresentação

Esse trabalho mostra os passos necessários para desenvolver um sistema de aquisição de dados completo com telemetria e gravador de dados em um protótipo de Fórmula SAE, além de mostrar como conseguir utilizar esses dados de maneira a melhorar a *performance* do veículo.

O trabalho irá descrever o desenvolvimento do sistema elétrico eletrônico do protótipo E17 usado pela equipe EESC USP Formula SAE no ano de 2019. Com esse sistema, é possível controlar a distribuição de energia para o carro, monitorar sinais em tempo real pela telemetria, analisar dados mais complexos por meio de um cartão de memória, gerenciar os atuadores pneumáticos e elétricos para controle de troca de marcha e abertura da asa traseira, entre outras funções que serão explicadas mais à frente.

1.2 Motivação e Objetivos

Em ambientes de competições automotivas, é de extrema importância entender como o carro se comporta para poder extrair o maior desempenho possível, além de monitorar sinais importantes, como temperaturas e pressões, em tempo real e garantir um sistema elétrico robusto que não falhe durante uma corrida.

No Brasil, competições de Fórmula SAE ocorrem uma vez ao ano, assim é preciso garantir que o carro não sofra nenhuma falha durante as provas, para que o trabalho feito durante o ano não seja perdido. Sistemas elétricos são um ponto de falha comum em competições de Fórmula SAE, algo que pude observar durante 4 competições, por serem uma área crítica para o funcionamento do veículo, mas bastante complexa no que diz respeito aos conhecimentos teóricos e práticos necessários.

Assim, foi definido um objetivo de desenvolver o um sistema que preza pela simplicidade e confiabilidade. Um exemplo de decisão para diminuir as chances de falha, foi o uso de relés de estado sólido no lugar de relés eletromecânicos, por possuírem um tempo de vida maior. Porém, o uso desses dispositivos requer um estudo teórico maior para garantir que o seu sistema consiga fornecer energia suficiente estando sempre dentro dos parâmetros necessários.

É de fundamental importância para a equipe entender o funcionamento dos diferentes subsistemas do protótipo, e isso pode ser feito pela aquisição de dados. Diferentes sensores posicionados ao longo do protótipo permitem obter informações que podem ser usadas para validar e melhorar projetos, realizar mudanças nos ajustes de *setup*, e comparar diferentes pilotos a fim de extrair o máximo de desempenho.

Assim, o principal objetivo desse sistema, é de capturar os dados dos vários sensores espalhados no carro, com uma frequência suficientemente alta, e de maneira confiável, para permitir o uso posterior de maneira correta. Uma vez capturados, é preciso salvar esses dados, e isso é feito de duas maneiras: envio por telemetria e gravação em um cartão de memória. O sistema de telemetria do carro permite o envio dos dados em tempo real para permitir o monitoramento de sinais críticos e comportamento dos outros sensores, mas permite também o salvamento dos dados no computador que está fazendo o recebimento dos dados. Porém, como o envio de dados pelo ar ocasiona em perdas de transmissão, existe um cartão de memória no carro que salva todos os dados diretamente nele. Assim, quando o carro para, é possível tirar esse cartão e descarregar os dados para ter acesso às informações completas do carro. Esse sistema é conhecido como *datalogger*.

A análise de dados permite à equipe utilizar os dados salvos pelo sistema eletrônico de maneira a melhorar o desempenho do carro. A partir dela é possível entender melhor o comportamento do veículo, analisar e comprar os pilotos e verificar a integridade do sistema como um todo. A análise de dados permite um entendimento maior do protótipo, fazendo assim com que a equipe consiga de melhorar seu desempenho e chegar mais perto do potencial máximo do carro.

Além de realizar a leitura e monitoramento do protótipo, também foi definido como objetivo a criação de um sistema responsável pela troca de marchas, acionamento da embreagem, e abertura dos *flaps* da asa traseira.

Os controles de troca de marcha e acionamento da embreagem servem para auxiliar o piloto e permitir a padronização dos movimentos, reduzindo assim a variância entre as ações recorrentes de erro humano, e permitem um ganho significativo no tempo de volta. O controle da asa traseira tem como objetivo reduzir o arrasto aerodinâmico em retas, e é controlado por um botão no volante, permitindo ao piloto uma aceleração e velocidades maiores, o que permite novamente um ganho significativo no tempo de volta.

1.3 Estrutura desse Documento

Esse documento é dividido nos seguintes capítulos: introdução, que apresenta o projeto e lista as motivações e objetivos; no Capítulo 2 falaremos sobre os conceitos teóricos, para familiarizar o leitor com conceitos necessários para o entendimento do projeto; no Capítulo 3 falaremos sobre os principais passos do desenvolvimento de *hardware* e *firmware*, tão quanto a implementação de sensores e atuadores; o Capítulo 4 tem como objetivo mostrar como os dados são de fato utilizados para melhorar o desempenho do veículo; e por fim o Capítulo 5 conclui e retoma os principais pontos do projeto e discute sua importância.

2 EMBASAMENTO TEÓRICO

Nesse capítulo serão apresentados os conceitos necessários para entendimento do projeto.

2.1 Fórmula SAE

A Fórmula SAE é uma competição estudantil que ocorre em vários países do mundo, sendo que no Brasil é organizada pela SAE Brasil, e ocorre uma vez por ano. As equipes, compostas por alunos de engenharia de uma mesma universidade, desenvolvem um protótipo ao longo do ano para participarem da competição, que contou com mais de 70 equipes em 2019 (SAE Brasil, 2020). A Figura 1 mostra o protótipo E16 utilizado na competição de 2018.

Figura 1: Protótipo da equipe EESC-USP Fórmula SAE



Fonte: Autor

A competição dura três dias, nos quais as equipes são submetidas a provas estáticas e dinâmicas. As provas estáticas servem para que os participantes apresentem com mais detalhes todo o desenvolvimento do projeto ao longo do ano. Existem três provas estáticas: *Design*, no qual os participantes apresentam todo o projeto desenvolvido ao longo do ano de suas respectivas áreas, e são avaliados por juízes especializados. Apresentação é a prova na qual a equipe deve apresentar um plano de negócio baseado no protótipo, e mostrar para juízes que se passam por possíveis investidores do projeto. Por fim, a prova de Custos e Manufatura exige que a equipe crie um relatório detalhando o custo de todas as peças do carro, além de um manual com todo o processo de montagem e desmontagem do veículo.

Já as provas dinâmicas testam diretamente a capacidade do protótipo. São elas, o *skidpad*, aceleração, *autocross* e o *endurance*. Na prova de *skidpad*, o carro precisa percorrer

um círculo com raio 15,25m a fim de avaliar sua aceleração lateral. A prova de aceleração consiste em uma reta de 75m, onde o carro larga parado e deve terminar o mais rápido possível. O *autocross* é feito em um circuito e a equipe tem 4 oportunidades para tentar fazer uma volta rápida. A prova de *endurance* é a que mais vale pontos na competição, e o carro deve percorrer aproximadamente 22km sem sofrer nenhuma quebra ou falha. Existe também a prova de eficiência, que utiliza o *endurance* para medir o consumo de combustível usado pelo carro, e a equipe que tiver a maior eficiência, obtém a maior pontuação.

Todas as provas possuem uma pontuação associada, e ao final da competição a equipe que somar mais pontos, ganha a competição. A Tabela 1 mostra as pontuações associadas a cada prova. Além disso, a equipe ganhadora da competição no Brasil, ganha o direito de competir no ano seguinte nos Estados Unidos.

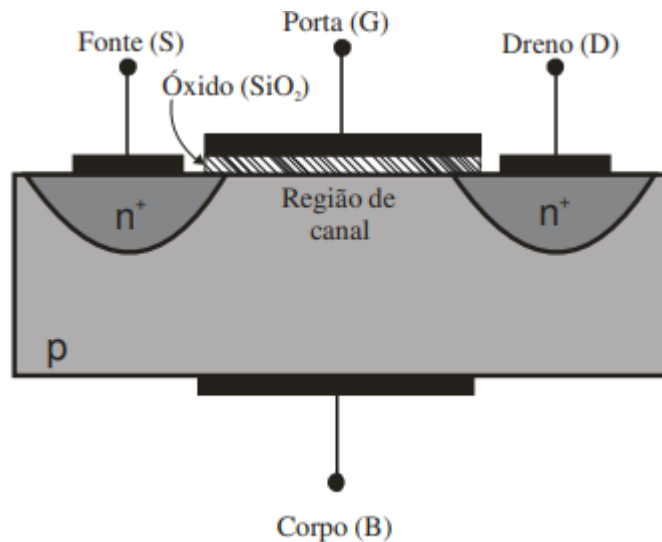
Tabela 1: Pontuação da competição

Provas Estáticas	Pontuação
Apresentação	75
Design	150
Custos e Manufatura	100
Provas Dinâmicas	Pontuação
Aceleração	100
Skidpad	75
Autocross	125
Eficiência	100
Enduro	275
Total de Pontos	1000

2.2 Transistores MOS

Transistores MOS, ou MOSFETs (*Metal-Oxide-Semiconductor Field Effect Transistor*) é um dos tipos de transistores mais usados no mercado. Sua aplicação vai desde processadores e placas de vídeo, até chaveamento de circuitos de alta potência. Sua estrutura segue a da Figura 2.

Figura 2: Estrutura transistor MOS

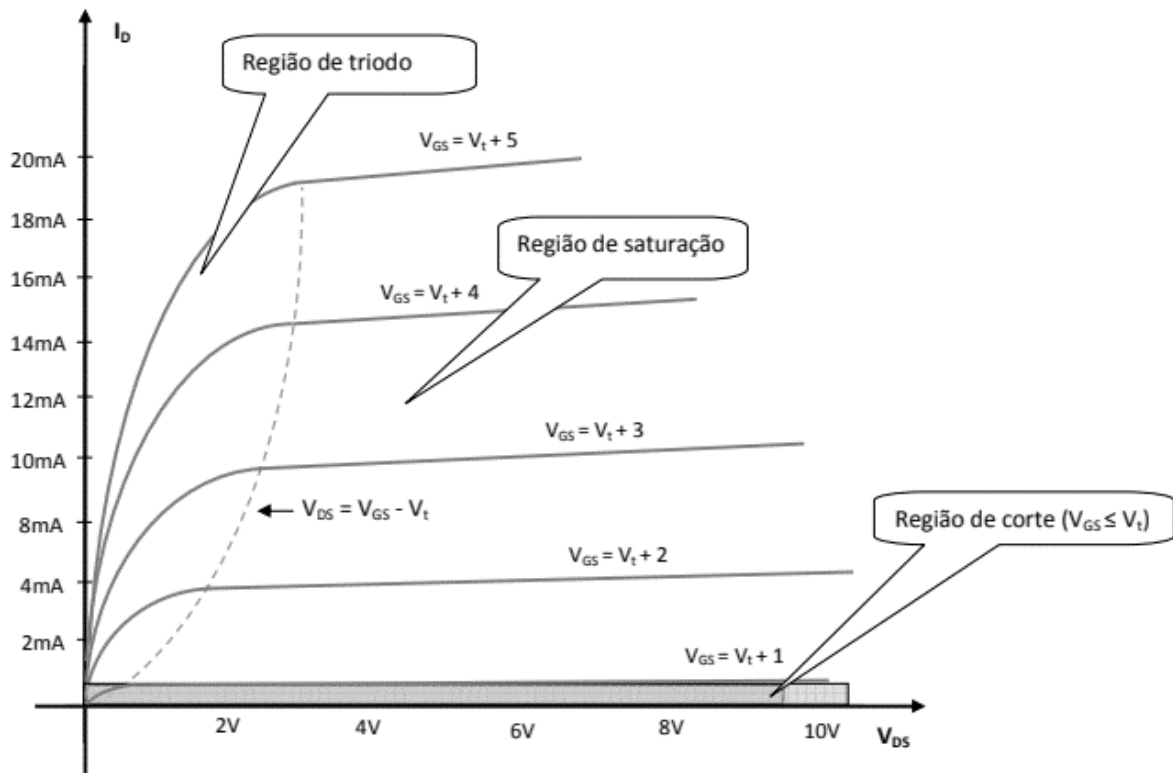


Fonte: Unicamp. 2020. Disponível em
<http://www.dsif.fee.unicamp.br/~elnatan/ee640/aula%20mosfet.pdf>

Ao aplicar uma diferença de tensão entre a Porta (*Gate*) e o Corpo (*Base*), um canal é criado entre a Fonte (*Source*) e o Dreno (*Drain*), permitindo assim a passagem de corrente. O tamanho desse canal, e a corrente suportada por ele, varia de acordo com a tensão entre a porta e o corpo. Assim, é possível manipular a corrente entre a fonte e o dreno alterando a tensão na porta.

Um transistor MOS pode operar em várias regiões diferentes, sendo elas região de corte, região de triodo e região de saturação, como mostra a Figura 3:

Figura 3: Regiões de operação transistor MOS



Fonte: Unicamp. 2020. Disponível em

<http://www.dsif.fee.unicamp.br/~elnatan/ee640/aula%20mosfet.pdf>

Como o interesse é apenas no uso de chaveamento de circuitos de potência, apenas a região de triodo é usada. Nessa região, a tensão entre fonte e dreno é baixa, assim a perda de potência no transistor é diminuída. Nessa região, temos a seguinte Equação 1

$$I_D = \mu_P C_{OX} \left(\frac{W}{L} \right) (V_{GS} - V_{TN}) V_{DS} \quad (1)$$

onde I_D é a corrente que passa pelo dreno, μ_P é a mobilidade elétrica, C_{OX} é a capacitância da Porta, W é a largura do transistor, L é o comprimento do transistor e V_{TN} é a tensão de *threshold*. Manipulando a equação, podemos saber a impedância do resistor no circuito pela Equação 2.

$$\frac{V_{DS}}{I_{DS}} = \frac{1}{\mu_P C_{OX} \left(\frac{W}{L} \right) (V_{GS} - V_{TN})} \quad (2)$$

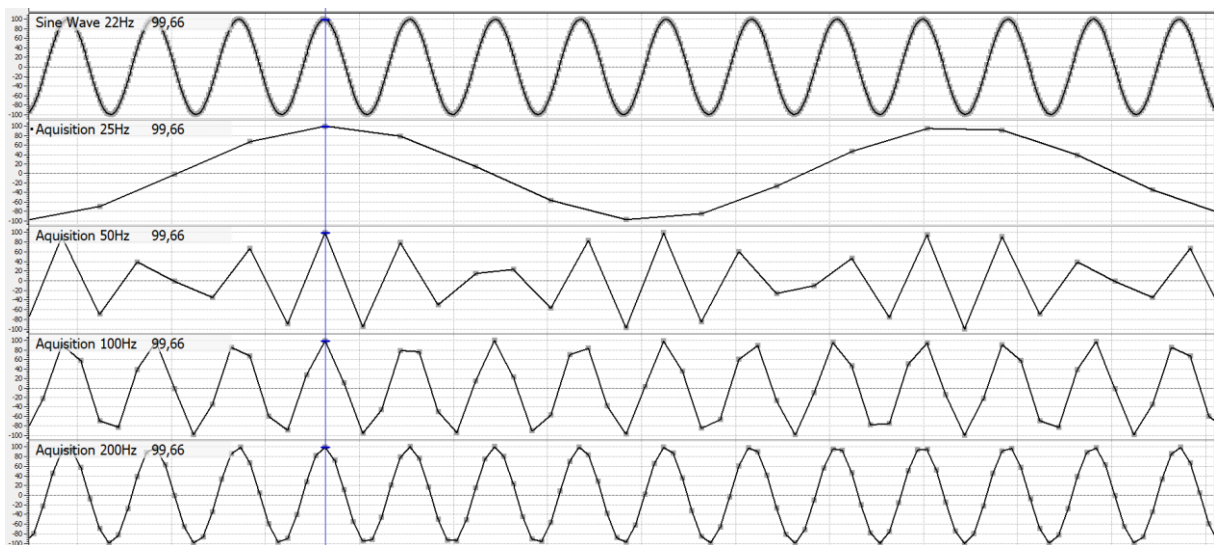
É possível perceber então que o MOSFET funciona como um resistor com resistência variável, e é possível obter a menor resistência aumentando a tensão entre a porta e a fonte, diminuindo assim a potência dissipada.

2.3 Aquisição de sinais

O processo de capturar sinais analógicos envolve várias etapas. Primeiramente, é necessário um sensor para converter uma grandeza mecânica em tensão ou corrente elétrica. Após isso é necessário filtrar esse sinal para eliminar alguns dos ruídos indesejáveis e possibilitar sua reconstrução posteriormente. Um conversor ADC é responsável por transformar uma tensão arbitrária em um valor binário. Por fim, um microcontrolador efetua o salvamento ou transmissão do dado.

Temos, pelo teorema de Nyquist, que devemos capturar um sinal a uma frequência de aquisição discreta pelo menos duas vezes maior que a frequência máxima do sinal analógico, para que seja possível reconstruir o sinal posteriormente. Caso contrário, o sinal original é perdido e o sinal capturado pode representar algo completamente diferente do sinal original. Na Figura 4 é possível ver um exemplo:

Figura 4: Aquisição de um sinal senoidal a várias frequências diferentes



Fonte: Autor

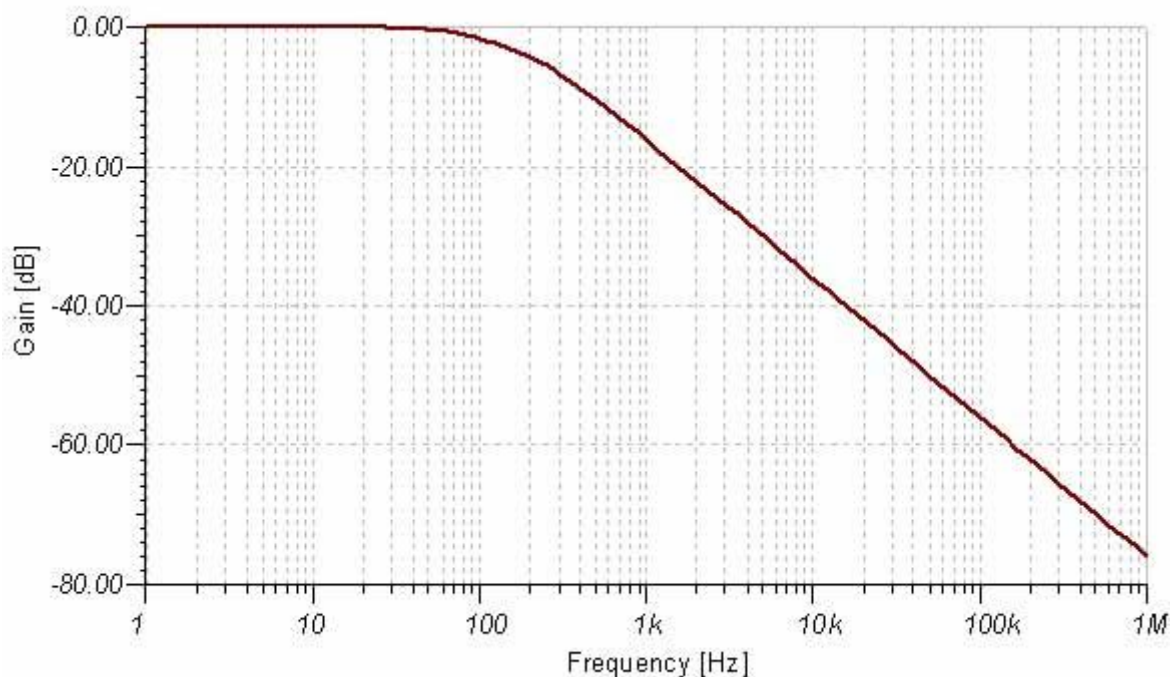
O primeiro sinal é uma senoide com frequência de 22Hz, e em seguida, sinais reconstruídos com frequências de amostragem de 25Hz, 50Hz, 100Hz e 200Hz. É possível perceber que a amostragem a 25Hz cria um sinal que não corresponde com o sinal original.

A aquisição a 50Hz, por mais que tenha uma frequência de amostragem maior que o dobro da frequência do sinal, não representa o sinal muito bem, e para efeitos de análise acaba dificultando bastante. Com 100Hz, já é possível observar um sinal mais uniforme, e com 200Hz, o sinal pode ser reconstruído de uma maneira bem fiel ao sinal original.

Com isso, é possível perceber claramente que quanto maior a frequência de aquisição, melhor será o sinal obtido, e assim, sua análise.

Um outro problema na aquisição é garantir que frequências mais altas não interfiram no sinal obtido, assim são utilizados filtros passa-baixa para limitar a banda de frequências que serão amostradas. Ao dimensionar um filtro, deve-se considerar a frequência de amostragem e de excitação do sensor. Um filtro RC de primeira ordem pode ser criado com um resistor e um capacitor, e possui o seguinte comportamento como mostra a Figura 5.

Figura 5: Amplitude do diagrama de bode para filtro passa-baixa



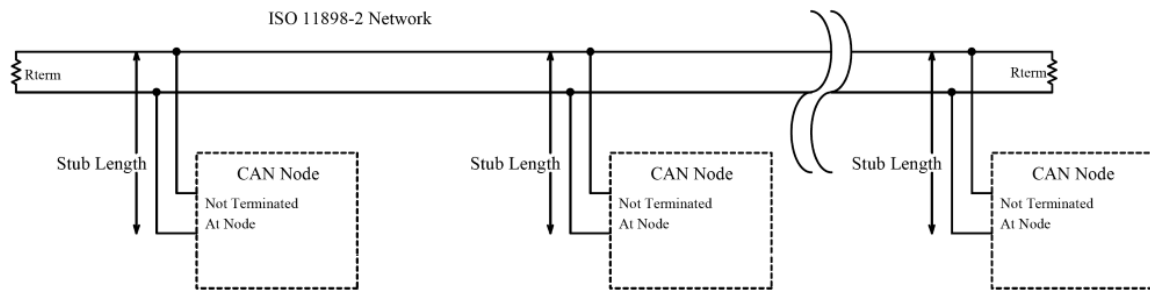
Fonte: *University of Surrey*. 2020. Disponível em

<http://www.ee.surrey.ac.uk/Projects/CAL/frequency-response/activelp.html>

2.4 Barramento CAN

O barramento CAN é um protocolo de comunicação desenvolvido pela *Robert Bosch GmbH* com foco em aplicação automotiva. A implementação de uma rede CAN consiste em um par de fios trançados que percorre o chicote elétrico. Os módulos eletrônicos, ou nós, se conectam ao barramento e enviam e recebem mensagens por ele. A Figura 6 mostra um exemplo de distribuição dos componentes em uma rede CAN.

Figura 6: Exemplo barramento CAN

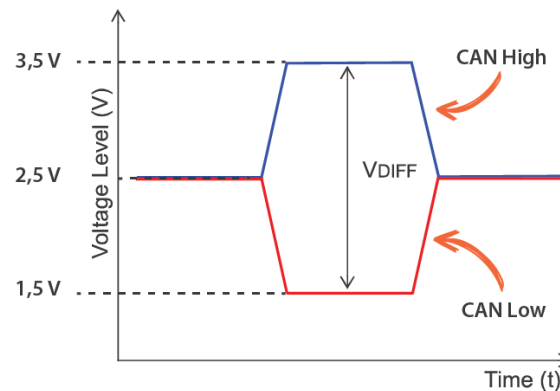


Fonte: Tek Eye. 2020. Disponível em
<https://tekeye.uk/automotive/can-bus-cable-wiring>

Para se adicionar um nó, uma emenda deve ser feita no barramento, que será conectada no nó. Existem várias recomendações ao implementar o barramento do ponto de vista elétrico. Primeiramente, os pontos do barramento devem finalizar em um resistor, como mostra a figura acima, tipicamente com valor de 120Ω . Isso serve para evitar a reflexão dos sinais elétricos. O comprimento total do barramento deve ser menor que um valor proporcional à taxa de transmissão. O comprimento de cada uma das emendas também deve respeitar um valor máximo proporcional a taxa de transmissão. O par de fios também deve ser trançado para reduzir possíveis interferências eletromagnéticas. A distância entre cada trança também tem um valor recomendado.

Uma das principais vantagens da rede CAN é sua alta robustez, ideal para automóveis, que costumam possuir um alto nível de ruídos. Um dos motivos por isso acontecer é o uso de um par diferencial. Os dois sinais da rede CAN são chamados de *CAN High* e *CAN Low*, e cada bit de transmissão pode ter dois estados. Recessivo, no qual ambos os sinais possuem uma tensão igual de aproximadamente 2,5V, e dominante, na qual o sinal baixo se aproxima de 1,5V e o alto de 3,5V. Assim, no estado recessivo a diferença de tensão entre os sinais é 0, e no dominante 2V. A Figura 7 mostra um exemplo desses sinais.

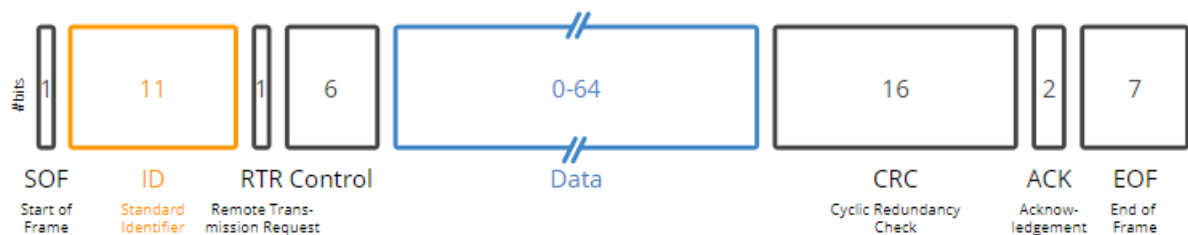
Figura 7: Níveis de tensão no barramento CAN



Fonte: *KMP Drivetrain Solutions*. 2020. Disponível em <https://www.kmpdrivetrain.com/paddleshift/practical-tips-can-bus/>

Como o importante para o barramento é a diferença de tensão entre os dois sinais, e estes são trançados, a interferência eletromagnética acaba sendo reduzida drasticamente. Uma outra medida que contribui para a robustez do sistema é o tamanho curto das mensagens. Cada mensagem manda apenas 8 bytes de dados. Porém, cada mensagem enviada possui a seguinte estrutura, como mostra a Figura 8:

Figura 8: Estrutura de uma mensagem CAN



Fonte: *CSS Eletronics*. 2020. Disponível em <https://www.csselectronics.com/screen/page/simple-intro-to-can-bus/language/en>

O bloco *SOF* inicial demarca o início da mensagem. O bloco seguinte representa o identificador da mensagem. Cada mensagem CAN possui um identificador que é usado para identificar o que cada *byte* do bloco *Data* representa. Esse identificador pode ser do tipo padrão, que ocupa 11 bits, ou estendido, que ocupa 29 bits. É importante ressaltar que mensagens do tipo padrão e estendido possui estruturas levemente diferentes, mas podem ser compartilhadas no mesmo barramento sem nenhum problema. O bloco *RTR* é referente ao modo de transmissão remoto, mas esse modo não será visto nesse documento. O bloco *Data* possui 8 *bytes* e

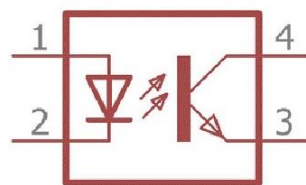
representa o conteúdo da mensagem em si. O bloco *CRC* serve como redundância para verificar se a mensagem foi enviada corretamente, aumentando ainda mais a robustez do protocolo. O bloco *ACK* indica que um nó recebeu os dados corretamente, e por fim, o bloco *EOF* indica o fim da mensagem.

Todas as mensagens podem ser enviadas por qualquer um dos nós conectados, e caso duas mensagens tentem ser transmitidas ao mesmo tempo, aquela com identificador menor terá a prioridade. Todos os nós têm a possibilidade de receber todas as mensagens, mas usualmente filtros são aplicados para que apenas as mensagens dos identificadores relevantes sejam recebidas, reduzindo assim a uso total do processador. É importante mencionar também que um mesmo barramento deve operar uma mesma taxa de transmissão, que deve ser configurada em cada um dos módulos. Normalmente, essa taxa é de 1Mbps.

2.5 Opto-isolador

Opto-isoladores, ou acopladores ópticos, são componentes elétricos compostos por um LED, normalmente infravermelho, e um fototransistor. Sua principal função é isolar eletricamente dois circuitos distintos, porém permitir a passagem de um sinal de entrada. Isso ocorre pois o LED produz um sinal luminoso proporcional a sua corrente, o que ativa o fototransistor e permite a passagem de corrente do outro circuito. A Figura 9 mostra o esquemático de um opto-isolador.

Figura 9: Esquemático opto-isolador



Fonte: *Components 101*. 2020. Disponível em

<https://components101.com/articles/what-is-optocoupler-and-how-it-works>

No exemplo da figura, os pinos 1 e 2 são responsáveis pela ativação do LED, enquanto os pinos 3 e 4, representam o emissor e coletor, respectivamente, do fototransistor. Uma das principais utilidades desses dispositivos é de separar circuitos de alta tensão com circuitos de baixa tensão, pois como não há nenhum condutor elétrico entre os pinos de entrada com os de saída, não há perigo de um circuitar danificar o outro.

3 DESENVOLVIMENTO

Nesse capítulo será mostrado os principais passos e considerações do desenvolvimento do projeto. É importante notar que o foco não é mostrar um passo a passo exato de todo o processo, mas sim de mostrar ao leitor os principais aspectos do desenvolvimento.

3.1 Desenvolvimento de Hardware

O primeiro passo do desenvolvimento é o desenho das placas de circuito impresso. Ao todo foram produzidos cinco modelos diferentes, porém todos com as mesmas dimensões e conectores, para padronizar o encapsulamento.

O desenho das placas é feito em um software especializado, onde se começa com o esquemático dos circuitos, e em sequência é feito o posicionamento dos componentes e o desenho das trilhas.

Os cinco modelos de placas desenvolvidos foram:

- Módulo de Aquisição Principal:
 - 7 entradas analógicas;
 - 2 entradas digitais;
 - Cartão SD para salvamento dos dados.
- Módulo de Aquisição Secundário:
 - 8 entradas analógicas.
- Módulo de Controle:
 - 12 entradas digitais;
 - 3 saídas de alta corrente;
 - 2 saídas digitais.
- Módulo Transmissor:
 - Transmissor *XBee*;
 - Módulo GPS *NEO 6M*.
- Módulo de Distribuição de Energia:
 - 6 saídas de alta corrente;
 - Entradas digitais para controle de acionamento.

3.1.1 Circuitos em comum

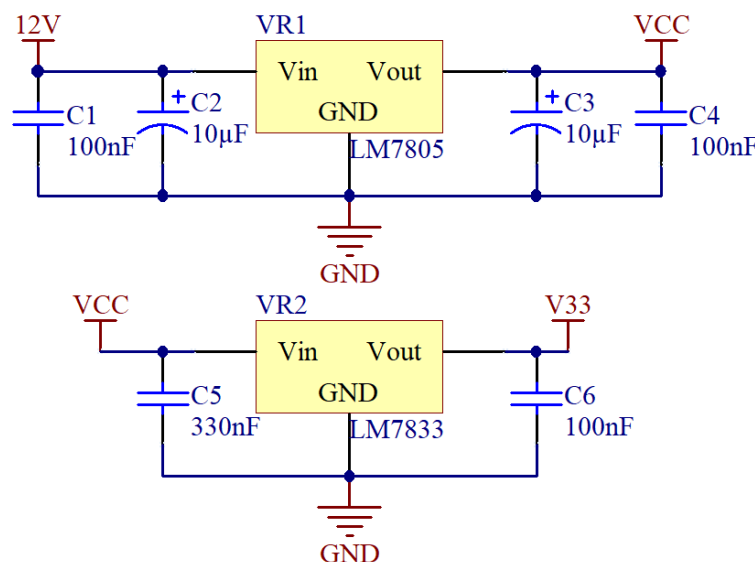
Todos os módulos possuem itens em comum. São eles reguladores de tensão, circuito para comunicação CAN, um microcontrolador e um conector automotivo.

3.1.1.1 Regulador de tensão

A fonte de energia elétrica em carros é composta normalmente por uma bateria de 12V, que é carregada por um alternador alimentado pelo motor a combustão. Porém, todos os componentes das placas trabalham com 5V ou 3,3V, sendo assim necessário reduzir essa tensão de maneira estável.

Felizmente, existem várias soluções comerciais acessíveis para realizar essa função. Os componentes escolhidos foram os chips LM7805 e LM7833, por serem facilmente acessíveis, serem de fácil implementação e fornecerem uma corrente de 1A, o que é mais do que o suficiente para cada módulo. A Figura 10 mostra o circuito implementado:

Figura 10: Circuito dos reguladores de tensão



Fonte: Autor

No primeiro circuito, a tensão de 12V da bateria é convertida em 5V (nó VCC) usando o LM7805. Os capacitores em paralelo seguem a recomendação do fabricante e servem como filtros para estabilizarem as tensões. Caso a fonte 12V sofra uma queda temporária, os capacitores irão suprir a carga necessária para deixar a tensão estável, até que a fonte se estabilize novamente. Os capacitores de 100nF servem para quedas de menor duração, pois conseguem fornecer uma carga mais rapidamente, enquanto os de 10µF servem para suprir uma maior carga caso necessário.

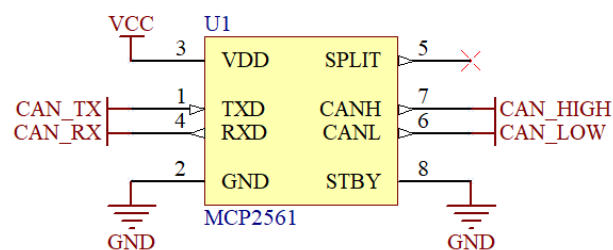
O circuito seguinte funciona com o mesmo princípio. O LM7833 faz a conversão de 5V para 3.3V (nó V33). Os capacitores seguem novamente a recomendação dos fabricantes.

Como a redução de 5V para 3.3V é menor do que 12V para 5V, e a fonte de 5V já está estabilizada com os capacitores do primeiro circuito, não é necessário usar capacitores de 10 μ F.

3.1.1.2 Interface CAN

O barramento CAN é a maneira de todos os módulos se comunicarem no carro, assim todos eles possuem um circuito dedicado com essa função. É preciso usar um CI que faça a interface do microcontrolador com o barramento em si. O CI usado foi o MCP2561, e a Figura 11 mostra sua implementação.

Figura 11: Implementação do MCP2561



Fonte: Autor

Os pinos 2 e 3 são referentes a alimentação do componente. O pino 8 faz a seleção do modo normal e em espera, como apenas o modo normal é usado, ele é aterrado. O pino 5 é apenas uma referência de tensão que equivale a $VCC/2$, mas para nossa aplicação não é utilizado. Os pinos 1 e 4 (*TXD* e *RXD*) são conectados direto ao microcontrolador, enquanto os pinos 6 e 7 correspondem aos sinais *CAN High* e *CAN Low*, e são conectados direto no barramento do carro.

3.1.1.3 Microcontrolador

O microcontrolador é o cérebro da placa. Foi decidido usar a família STM32, fabricado pela *STMicroelectronics*, devido a sua facilidade de implementação, alta quantidade de exemplos e documentação na internet, facilidade na criação do *firmware* (será visto mais a frente) e por questões relacionadas a patrocínio.

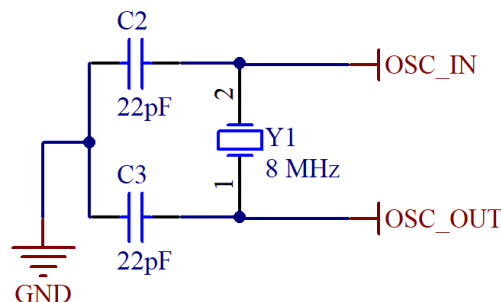
O microcontrolador escolhido foi o STM32F446, que possui uma memória *flash* de 512kB e SRAM de 128kB. Sua frequência máxima de *clock* é de 180MHz, possui 48 pinos de

IO, 15 entradas analógicas, além de possuir periféricos como ADC (x3), DMA, CAN (x2), I2C (x3), SPI (x3) e UART (x6).

A primeira implementação de um microcontrolador dessa família foi utilizando um modelo menos potente em uma placa estilo *shield* com todos os componentes necessários já implementados. Porém, um membro da equipe da eletrônica foi responsável por realizar a implementação do modelo mencionado acima, com maior poder de processamento e mais portas GPIO, diretamente na placa.

O primeiro passo da implementação é o desenho dos circuitos obrigatórios, como o cristal oscilador, alimentação, capacitores para estabilização da alimentação e circuito de *reset*. Começando com o cristal oscilador, é possível ver o circuito implementado na Figura 12. Os capacitores C2 e C3 são usados conforme recomendações do fabricante e servem para completar o circuito oscilador. A frequência do cristal é de escolha do usuário, mas o valor de 8MHz é comumente usado nesse modelo por permitir a frequência máxima de *clock*.

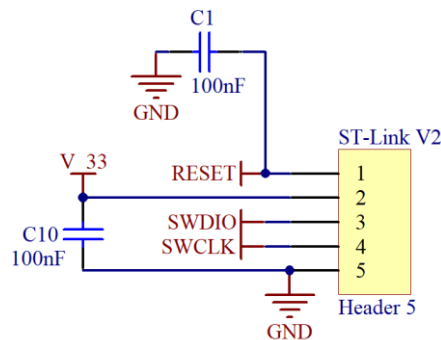
Figura 12: Circuito oscilador



Fonte: Autor

Passando então para o circuito que habilita a programação do microcontrolador, ele pode ser visto na Figura 13. O *Header 5* é por onde o programador se conecta na placa. Ele também consegue alimentar o microcontrolador com 3.3V, e para melhorar a estabilidade dessa fonte, existe um capacitor entre 3.3V e 0V. Os nós *RESET*, *SWDIO* e *SWCLK* vão direto para entradas do micro controlador. O pino *RESET* é responsável por reiniciar o micro quando um nível lógico baixo é aplicado. Para evitar possíveis reinícios não desejados, um capacitor de desacoplamento é inserido entre o nó e o terra. Junto com um resistor de *pullup* que se encontra dentro do microcontrolador, o capacitor funciona como um filtro passa-baixa que estabiliza a tensão evita que ruídos possam gerar um nível lógico baixo. Vale ressaltar que esse circuito normalmente é utilizado junto com um botão que reinicia o microcontrolador manualmente. Porém nesse caso o reinício é feito apenas pelo programador em si.

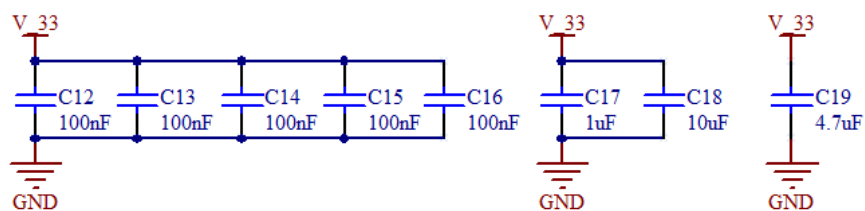
Figura 13: Circuito do programador



Fonte: Autor

Outro aspecto muito importante na implementação do microcontrolador é o uso correto de capacitores de desacoplamento nos pinos de alimentação. Placas de circuito impresso costumam possuir bastante ruído, e para conseguir manter a tensão de alimentação do microcontrolador estável, capacitores de desacoplamento são usados. No caso de uma queda no sinal da alimentação, os capacitores conseguem fornecer energia para o microcontrolador de maneira constante. São usados diferentes valores de capacitores para filtrarem diferentes tipos de ruídos.

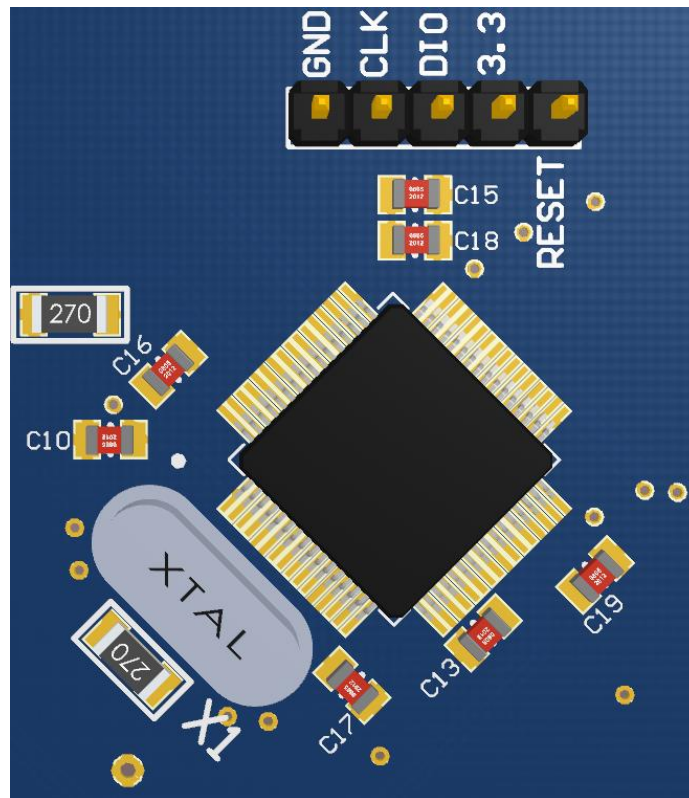
Na implementação do microcontrolador, os seguintes capacitores de desacoplamento são usados, como mostra a Figura 14. Cada capacitor tem um local no circuito impresso indicado pelo manual do fabricante.

Figura 14: Capacitores de *decouple*

Fonte: Autor

A Figura 15 mostra o microcontrolador com alguns dos componentes mencionados em volta dele. É possível ver o *header* do programador, o cristal oscilador, e os capacitores de desacoplamento. Também é válido mencionar que na parte de baixo da placa existem outros capacitores.

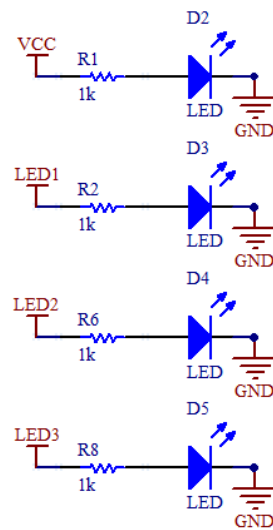
Figura 15: Implementação do microcontrolador



Fonte: Autor

Uma das desvantagens de se trabalhar com circuitos embarcados que não possuem um visor é a dificuldade de depuração em caso de problemas. Pensando nisso, todos os módulos possuem LEDs implementados para facilitar na depuração em casos de problemas e verificar, mesmo que parcialmente, se o módulo está funcionando corretamente. São usados entre 3 e 4 LEDs, sendo que um desses é conectado diretamente nos 5V para verificar se a placa está sendo alimentada, e os outros em saídas do microcontrolador, sendo que um desses quase sempre é configurado para piscar a cada 100ms, e os outros configurados para piscar em ações específicas para cada módulo, como por exemplo uma troca de marcha. A implementação desses LEDs pode ser vista na Figura 16. Os nós *LED1*, *LED2* e *LED3* se conectam diretamente a saídas do microcontrolador.

Figura 16: Utilização de LED

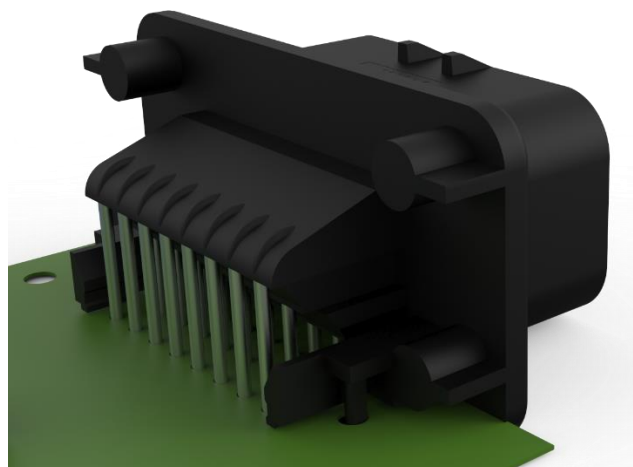


Fonte: Autor

3.1.1.4 Conector

Todos os módulos precisam de uma maneira de se conectar com o chicote elétrico, e em um ambiente de competição, é importante que essa conexão seja selada, resistente e confiável. É utilizado um conector *AMPSEAL* de 23 vias produzido pela *TE Connectivity*, como mostra a Figura 17. Com esse conector, é possível retirar e posicionar os módulos no carro sem se preocupar com mal contatos elétricos e problemas relacionados a entrada de água, desde que as especificações do fabricante sejam seguidas.

Figura 17: Conector automotivo

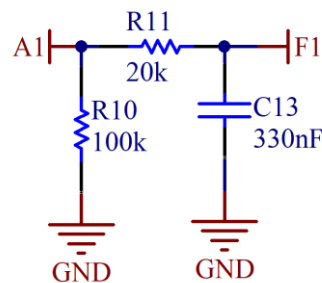


Fonte: Autor

3.1.2 Módulo de Aquisição

A principal característica dos módulos de aquisição é a presença de filtros passa-baixa ligados nas entradas analógicas do microcontrolador. Esses filtros têm a função de eliminar ruídos de alta frequência que podem interferir na leitura. Em alguns sensores específicos, o filtro também pode eliminar sinais de alta frequência que sejam acima da metade da frequência de aquisição. Os filtros utilizados são filtros RC de primeira ordem com uma frequência de corte de aproximadamente 25Hz, uma vez que a frequência de aquisição mínima da placa é de 50Hz. O circuito usado pode ser visto na Figura 18.

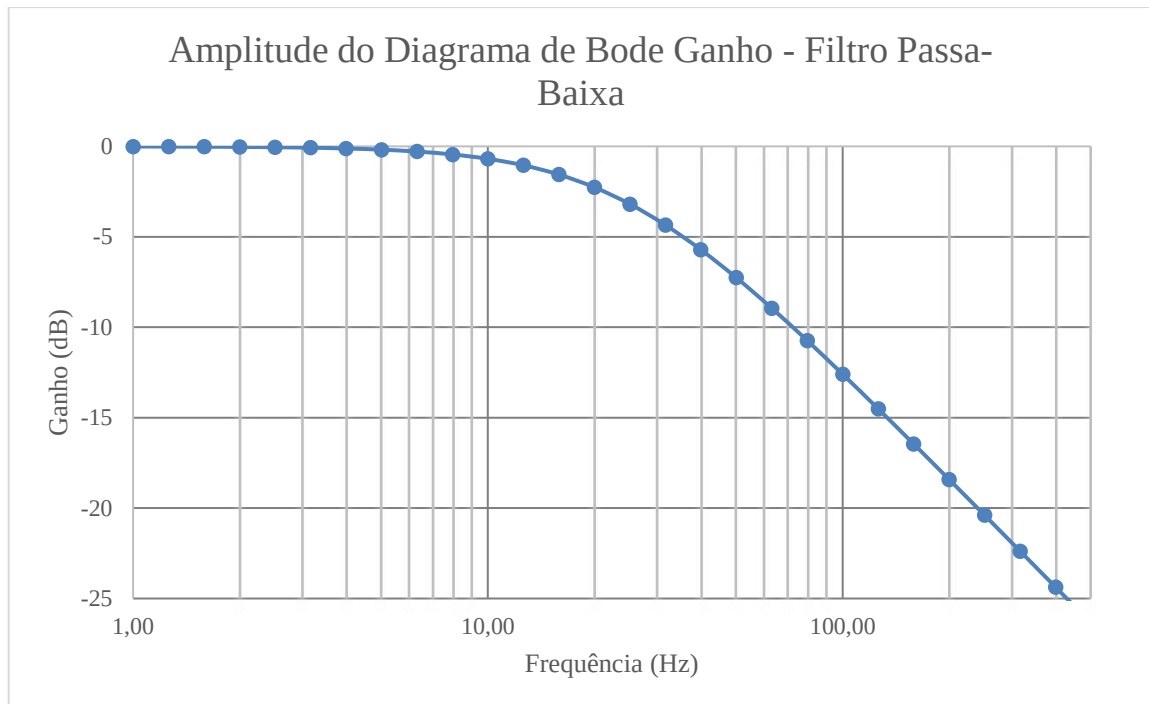
Figura 18: Filtro passa-baixa



Fonte: Autor

O nó *A1* representa a entrada do sinal, enquanto o nó *F1* um pino de leitura analógica no microcontrolador. O resistor *R10* serve como *pull-down* para caso nenhum sensor esteja conectado, a leitura seja de 0V e não fique flutuando. Os resistores *R11* e *C13* fazem parte do filtro passa-baixa. Para calcular a frequência de corte, temos que $H(s) = \frac{Y(s)}{X(s)} = \frac{1/sC}{R + 1/sC} = \frac{1}{sRC + 1} = \frac{151,5}{s + 151,5}$. Temos então que a frequência de corte é de 151,5 rad/s. Convertendo para Hertz, temos então um valor de 24,1Hz. Temos na Figura 19 a amplitude do diagrama de Bode do circuito acima simulado.

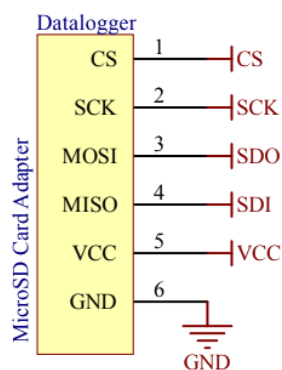
Figura 19: Amplitude do diagrama de bode filtro passa-baixa



Fonte: Autor

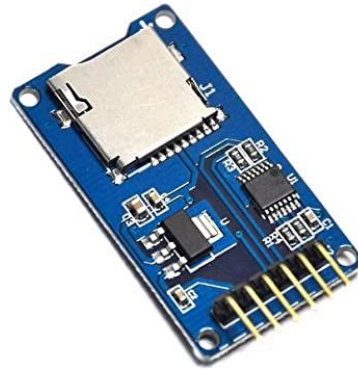
A módulo de aquisição principal também é responsável pelo salvamento dos dados em um cartão SD. Foi utilizado um adaptador que faz a comunicação do cartão SD com o microcontrolador usando o protocolo SPI. Sua implementação na placa é bem simples. Podemos ver na Figura 20 o circuito e na Figura 21 o adaptador em si.

Figura 20: Circuito do adaptador cartão SD



Fonte: Autor

Figura 21: Adaptador cartão SD



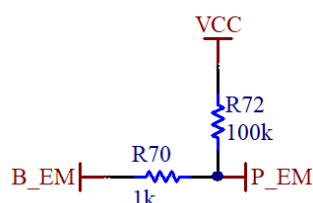
Fonte: Amazon. 2020. Disponível em

<https://www.amazon.com/Storage-Memory-Shield-Module-Arduino/dp/B01IPCAP72>

3.1.3 Módulo de Controle

A principal característica do módulo de controle é o alto número de entradas e saídas digitais. Quando se trata de entradas digitais, é preciso tomar alguns cuidados para proteger o microcontrolador. Caso a tensão da entrada seja no máximo de 5V, é possível usar apenas alguns resistores para fazer a proteção do circuito. Porém, caso esses sinais sejam 12V, ou sejam mais propícios a falhas que exponham o sinal a tensões maiores de 12V, é interessante isolar os circuitos do microcontrolador e da origem do sinal de maneira que esses sinais não se conectem por nenhum componente elétrico.

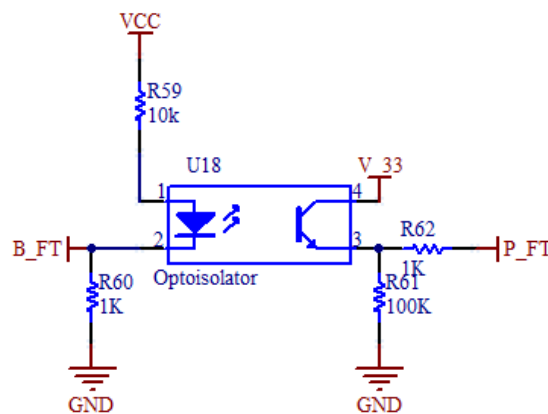
O tipo mais simples de entrada digital são sinais normalmente abertos que aterram em um momento determinado. São usados 2 resistores apenas. Um *pullup* para que o sinal fique em nível lógico alto quando abertos, e um resistor em série com o microcontrolador para proteger de uma possível tensão alta que seja aplicada involuntariamente. Segue na Figura 22 um exemplo desse circuito, onde B_EM é o sinal vindo do botão da embreagem, e P_EM a entrada do microcontrolador.

Figura 22: Entrada digital com *pullup*

Fonte: Autor

Porém, ao trabalhar com entradas que funcionam com 12V, é interessante adicionar um opto-isolador entre o sinal e o microcontrolador. Na Figura 23 é possível ver um exemplo aplicado. O nó *B_FT* é uma saída da *ECU* que opera entre 0V-12V, e nó *P_FT* é uma entrada do microcontrolador. Quando *B_FT* for 0V, uma corrente irá seguir de *VCC* até *B_FT*, ligando assim o LED e acionando o fototransistor, colocando assim uma tensão de 3.3V com corrente limitada pelo resistor R62 na entrada do microcontrolador. Caso *B_FT* esteja em 12V, uma tensão reversa é aplicada no LED e ele não aciona. Assim, o resistor de *pull-down* R61 faz com que uma tensão de 0V seja aplicada na porta do microcontrolador. Assim, é possível isolar o circuito do microcontrolador com o da *ECU*.

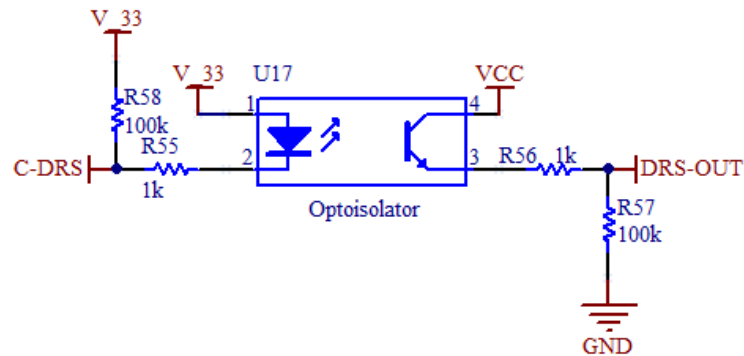
Figura 23: Entrada digital com opto-isolador



Fonte: Autor

Também é possível utilizar o mesmo princípio para as saídas do microcontrolador. No exemplo da Figura 24, *C-DRS* é uma saída PWM do microcontrolador, e *DRS-OUT* é o sinal de controle de um servomotor. Quando *C-DRS* estiver em 0V, o LED é ativado e a saída vai para *VCC*, com a corrente limitada pelo resistor R56. Quando *C-DRS* estiver em nível lógico alto, ou em alta impedância, o LED não acionará e a saída ficará em terra pelo resistor de *pull-down* R57.

Figura 24: Saída digital com opto-isolador

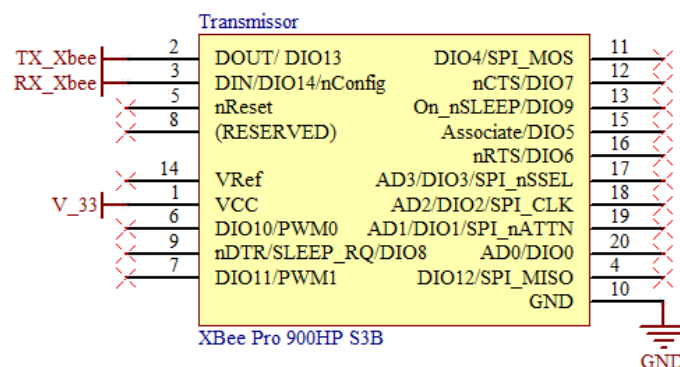


Fonte: Autor

Existem, porém, 3 saídas nesse módulo que necessitam de uma corrente maior. Nesse caso, é utilizado um circuito parecido com o acima, mas com a adição de um transistor de potência na saída. Dessa maneira, o sinal da saída continua isolado do microcontrolador, mas como o fototransistor dentro do optoisolador não fornece a corrente necessário, o transistor é usado para amplificar essa corrente.

3.1.4 Módulo Transmissor

O módulo transmissor consiste em dois componentes principais: um transmissor *XBee Pro 900HP* e um módulo GPS *NEO-6M*. Ambos utilizam comunicação serial para fazer a interface com o microcontrolador e possuem circuitos simples de implementar. A Figura 25 mostra o circuito do transmissor, e a Figura 26 mostra o modelo utilizado.

Figura 25: Circuito transmissor *XBee Pro 900HP*

Fonte: Autor

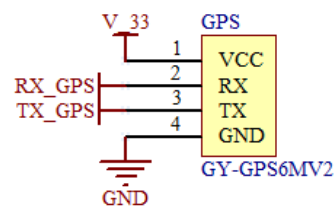
Figura 26: XBee Pro 900HP



Fonte: Digi. 2020. Disponível em <https://www.digi.com/products/embedded-systems/digi-xbee/rf-modules/sub-1-ghz-rf-modules/xbee-pro-900hp>

O módulo é alimentado com 3.3V e apenas dois pinos de comunicação são usados, que vão direto para o microcontrolador. O GPS possui uma implementação parecida, como é possível ver na Figura 27. A Figura 28 mostra o modelo utilizado.

Figura 27: Circuito GPS



Fonte: Autor

Figura 28: GPS *u-blox NEO-6M*



Fonte: Eletrodex. 2020. Disponível em

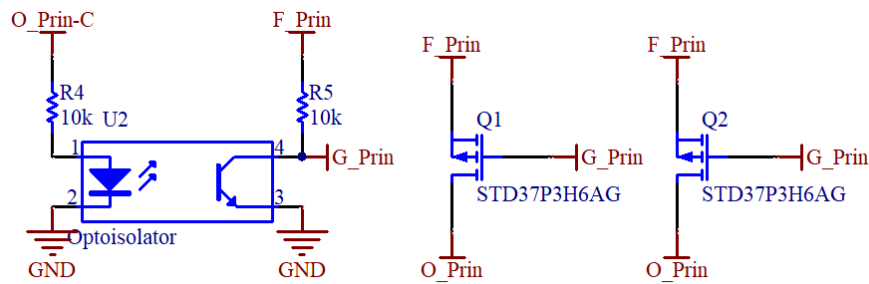
<https://www.eletrodex.com.br/gy-neo6mv2-modulo-gps-com-antena.html>

3.1.5 Módulo de Distribuição de Energia

O módulo de distribuição é o único dessa lista que é crítico para o funcionamento do protótipo, e qualquer problema que ocorra com ele pode facilmente acabar no desligamento do motor ou até mesmo em danos permanentes. Pensando nisso, embora o módulo possua um microcontrolador, esse não influencia nos sistemas críticos do carro e não é necessário para seu funcionamento. O módulo contém 6 saídas de potência que alimentam os outros módulos, as duas ventoinhas do arrefecimento, a bomba de combustível, a injeção eletrônica, e por fim uma saída que serve para vários sistemas como sensores 12V, bico injetor e a bobina de ignição.

Cada saída é composta por 2 *MOSFETs* tipo P em paralelo por redundância e um circuito com optoisolador para acionamento da porta dos transistores. A Figura 29 mostra o esquemático de uma das saídas pode ser vista a seguir. O nó *F_Prin* está ligado diretamente no fusível correspondente e é conectado pela chave externa na bateria. *O_Prin-C* é um sinal que vem da chave secundária de dentro do *cockpit* e é responsável por ativar todas as saídas, com exceção da saída que alimentam os módulos da aquisição de dados. Quando *O_Prin-C* está em 12V, o optoisolador ativa e o pino *G_Prin* vai para terra, ativando assim ambos os transistores, que conectam *F_Prin* em *O_Prin*, que é a saída da placa que alimenta diretamente os componentes. Quando *O_Prin-C* está aberto, *G_Prin*, a porta do transistor, fica em 12V e o transistor fica desligado.

Figura 29 Circuito de uma saída de potência



Fonte: Autor

Todas as saídas também possuem um LED no dreno para indicar visualmente de maneira fácil se a saída está funcionando de maneira correta ou não, o que facilita bastante o processo de depuração. Além disso, os transistores relacionados a ventoinha, possuem um circuito especial que permite o uso de uma função PWM controlado pelo microcontrolador, que quando em uso, permite o funcionamento intermitente das ventoinhas de acordo com a temperatura da água do motor, permitindo assim um controle maior dessa temperatura.

3.2 Firmware

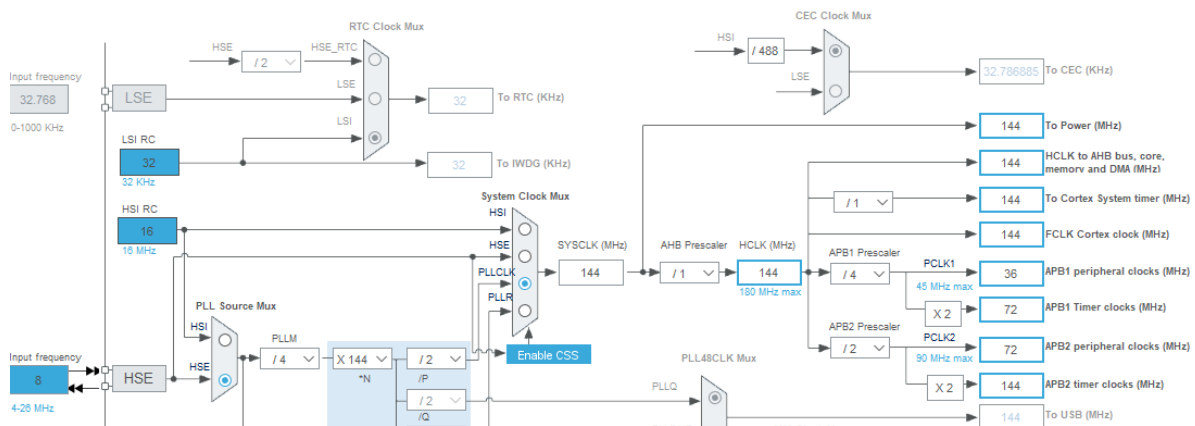
O desenvolvimento dos códigos dos microcontroladores é feito na plataforma *STMCubeIDE*, desenvolvido pelo mesmo fabricante do microcontrolador, ela possui ferramentas gráficas para auxiliar na criação do projeto e configuração de parâmetros. Essa ferramenta gráfica, chamada de *STMCubeMX*, gera um código fonte base que o usuário irá modificar por cima. No caso de mudanças nesses parâmetros, a ferramenta irá gerar novamente os códigos de base, mas sem interferir nos códigos inseridos pelo usuário.

Esse ambiente de trabalho também utiliza da biblioteca *HAL*, que se trata de um conjunto de API que permite criar uma camada de abstração entre o *hardware* e o *firmware*. Essa biblioteca permite o controle dos periféricos de uma maneira simplificada que permite uma portabilidade maior de código entre modelos diferentes de microcontroladores. A maior vantagem dessa biblioteca é a facilidade de controle dos periféricos. Por exemplo, para mudar o estado de um pino configurado como saída, basta utilizar a função *HAL_GPIO_WritePin*, que leva como parâmetros o pino em si e o estado desejado. Todas as execuções em baixo nível, como mudança dos registradores, são gerenciadas automaticamente.

3.2.1 Clock

O primeiro passo na criação de um projeto é a configuração do *clock*, que irá determinar a velocidade de execução do microcontrolador. Como já mostrado, é utilizado um cristal oscilador de 8MHz. Porém, é possível aumentar essa frequência utilizando multiplicadores, podendo chegar assim em uma frequência máxima de 144MHz. A Figura 30 mostra a parte dessa configuração. No canto esquerdo inferior temos a frequência do cristal, e seguindo seu caminho pelos multiplexadores, podemos ver que a frequência do *SYSCLK* é de 144MHz. Como nessa aplicação o alto desempenho é a prioridade, é optado por usar as frequências mais altas disponíveis.

Figura 30: Configuração *clock*



Fonte: Autor

3.2.2 Timer

Após a definição do *clock*, é possível iniciar a configuração dos periféricos. Começando com o *timer*, ele é responsável por permitir um controle com base no tempo nas ações a serem efetuadas. A ideia é ter uma variável que é incrementada a um período fixo, e utilizar essa variável para o controle de execução de tarefas que precisem ser executadas a uma frequência específica. Após habilitar um dos vários *timers* disponíveis, nesse caso o *TIM1*, e configurar a fonte de *clock* como o *clock* interno, o próximo passo é definir seu período de acionamento. Os parâmetros que controlam isso são o *Prescaler* e *Counter Period*. A fórmula que indica o período do timer pode ser vista na Equação 3.

$$T = \frac{1}{f_T} * (1 + PSC) * (1 + Counter Period)$$

Onde f_T é a frequência base do *timer* e PSC é o *Prescaler*. Pela Figura 30 anterior temos que a frequência do *TIM1* (nesse caso *APB2*) é de 144MHz, e o valor desejado de período é de 1ms. Assim, é possível escolher os valores adequados para que o período seja o desejado, porém ambas as variáveis são de 16 *bits* e possuem um valor máximo de 65.535, assim foi escolhido um valor de *PSC* de 35999 e *Counter Period* de 3, resultando assim em um período de 1ms.

Além de habilitar e configurar o *timer*, é preciso utilizá-lo de alguma maneira. Isso é feito através das interrupções. Toda vez que o *timer* reinicia, a cada 1ms, uma interrupção é acionada no microcontrolador, e o código atual que está sendo executada é parado temporariamente, um código relacionado a interrupção é executado, e após isso o programa retorna no ponto em que parou. Para desenvolver o código que será executada nessa interrupção, é preciso adicionar uma função como mostra o Código 1.

Código 1: Incrementação dos contadores

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if(htim->Instance == TIM1)
    {
        contador_100++;
        contador_msg_1++;
        contador_msg_2++;
        contador_can_1++;
        contador_can_2++;
        contador_verify++;
        contador_mensagem++;
    }
}
```

A função *HAL_TIM_PeriodElapsedCallback* é chamada durante a execução da interrupção, e pode ser implementada pelo usuário em qualquer arquivo. A vantagem desse método é de organizar todas as funções de interrupção da maneira que o usuário achar melhor. Nesse caso, todas as funções de interrupção estão em um arquivo criado para definições de funções. Assim, a cada 1ms, a função será executada e todas as variáveis mostradas serão incrementadas. É importante verificar a instância do *timer* que envia a interrupção no caso de mais de um *timer* estarem habilitados.

3.2.3 Rede CAN

A habilitação da rede CAN é bem simples, sendo primeiro necessário sua habilitação e configuração de taxa de transmissão, no caso de 1Mbps, por ser a mais rápida que o microcontrolador permite, além de ser a mesma da injeção eletrônica. Após isso, a interrupção de recebimento de mensagens é habilidade, caso a placa específica necessite.

Para poder começar a receber mensagens, é preciso configurar filtros de entrada. São 14 filtros disponíveis, sendo que cada um possui um identificador e uma máscara. Ao receber uma mensagem, o filtro irá aplicar a máscara ao identificador recebido (operação *AND*) e irá então comparar com o identificador do filtro. Se ambos forem iguais, a mensagem será aceita em uma das filas disponíveis, configurada no filtro. A implementação desse filtro pode ser vista no Código 2:

Código 2: Configuração da rede CAN

```
/*MENSAGENS CAN*/
void Configura_CAN()
{
    CAN_FilterTypeDef sFilterConfig;

    uint32_t filter_id = 0xD500, mask_id = 0xFF00;
    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
    sFilterConfig.FilterIdHigh = filter_id >> 13; // & 0xFFFF;
    sFilterConfig.FilterIdLow = (filter_id << 3) & 0xFFF8;
    sFilterConfig.FilterMaskIdHigh = mask_id >> 13; // & 0xFFFF;
    sFilterConfig.FilterMaskIdLow = (mask_id << 3) & 0xFFF8;
    sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    sFilterConfig.SlaveStartFilterBank = 13;

    HAL_CAN_ConfigFilter(&hcan1, &sFilterConfig);

    HAL_CAN_Start(&hcan1);
}
```

É possível ver que nesse caso o ID do filtro é de 0xD500 e a máscara 0xFF00. Isso significa que todas as mensagens recebidas com um identificador no formato 0xD5__ serão aceitas. Isso é feito pois todas as mensagens enviadas pelo sistema desenvolvido possuem 0xD5__ como parte do identificador. Na Tabela 2, é possível ver uma tabela com alguns dos dados enviados:

Tabela 2: Parte dos dados enviados na rede CAN

Módulo	Mensagem	Bit/s CAN	Frequência (Hz)	Mensagem	Byte	Descrição
Controle	25	1200	10	MSG_CONTROLE_SHIFTER	0	MARCHA
					1	NUM UPSHIFT
					2	NUM DOWNSHIFT
					3	TEMPO UPSHIFT
					4	TEMPO DOWNSHIFT
					5	CORTE CENTELHA
					6	FLAG IWDG
	26	1280	10	MSG_CONTROLE_ECU_1	0	RPM HIGH
					1	RPM LOW
					2	TPS
					3	IAT
					4	BATERIA HIGH
					5	BATERIA LOW
					6	ECT HIGH
	27	1280	10	MSG_CONTROLE_ECU_2	7	ECT LOW
					0	MAP HIGH
					1	MAP LOW
					2	PRESSAO COMB HIGH
					3	PRESSAO COMB LOW
					4	PRESSAO OLEO HIGH
					5	PRESSAO OLEO LOW
	28	960	10	MSG_CONTROLE_ECU_3	6	LAMBDA METER HIGH
					7	LAMBDA METER LOW
					0	VEL RODA FL
Transmissor	30	640	5	MSG_GPS_DATA1	1	VEL RODA FR
					2	VEL RODA RL
					3	VEL RODA RR
					0	POSIÇÃO SERVO
					1	POSIÇÃO ON
					2	POSIÇÃO OFF
					3	DRS COUNT
	31	640	5	MSG_GPS_DATA2	0	LATITUDE GRAUS
					1	LATITUDE MINUTOS
					2	LATITUDE SEGUNDOS HIGH
					3	LATITUDE SEGUNDOS LOW
					4	LONGITUDE GRAUS
					5	LONGITUDE MINUTOS
					6	LONGITUDE SEGUNDOS HIGH
					7	LONGITUDE SEGUNDOS LOW
	31	640	5	MSG_GPS_DATA2	0	HORA
					1	MINUTO
					2	SEGUNDO
					3	DIA
					4	MES
					5	ANO
					6	VELOCIDADE HIGH
					7	VELOCIDADE LOW

Em uma das placas, porém, é preciso fazer o recebimento dos dados da injeção eletrônica. Nesse caso, um segundo filtro é adicionado, utilizando um identificador e uma máscara que corresponda ao padrão utilizado pela ECU. Também é utilizada uma outra fila de recebimento, para diminuir a interferência que um sistema possa ter na leitura do outro.

Após configurar todo o recebimento de mensagens, é preciso implementar a função de *callback* da interrupção, que será chamada toda vez que a interrupção ocorrer, isto é, uma mensagem for recebida. O Código 3 mostra essa implementação:

Código 3: Recebimento de mensagens CAN

```
//Override
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    if(HAL_CAN_GetRxMessage(hcan,CAN_RX_FIFO0,&RxMessage, rxData)==HAL_OK)
    {
        Escrever_Dados_Datalogger(&RxMessage, rxData);
    }
}
```

Nesse exemplo, a função *HAL_CAN_GetRxMessage* irá passar o conteúdo da mensagem recebida para as variáveis *RxMessage* e *rxData*. *RxMessage* é uma *struct* que contém os parâmetros da mensagem e *rxData* o vetor que possui os dados em si. Após isso, a função *Escrever_Dados_Datalogger* irá escrever no cartão SD embarcado os dados da mensagem recebida.

Para realizar o envio de mensagens, o seguinte procedimento é feito, como mostra o Código 4:

Código 4: Envio de mensagem CAN

```
void Enviar_Mensagem_1()
{
    TxMessage.IDE = CAN_ID_EXT;
    TxMessage.RTR = CAN_RTR_DATA;
    TxMessage.ExtId = 0xD50A;
    TxMessage.DLC = 8;
    TxMessage.TransmitGlobalTime = DISABLE;

    txData[0] = ADC_BUFFER[ACCEL_X] >> 8;
    txData[1] = ADC_BUFFER[ACCEL_X] & 0xff;
    txData[2] = ADC_BUFFER[ACCEL_Y] >> 8;
    txData[3] = ADC_BUFFER[ACCEL_Y] & 0xff;
    txData[4] = ADC_BUFFER[ACCEL_Z] >> 8;
    txData[5] = ADC_BUFFER[ACCEL_Z] & 0xff;
    txData[6] = ADC_BUFFER[GYRO] >> 8;
    txData[7] = ADC_BUFFER[GYRO] & 0xff;

    HAL_CAN_AddTxMessage(&hcan1, &TxMessage, txData, &pTxMailbox);
}
```

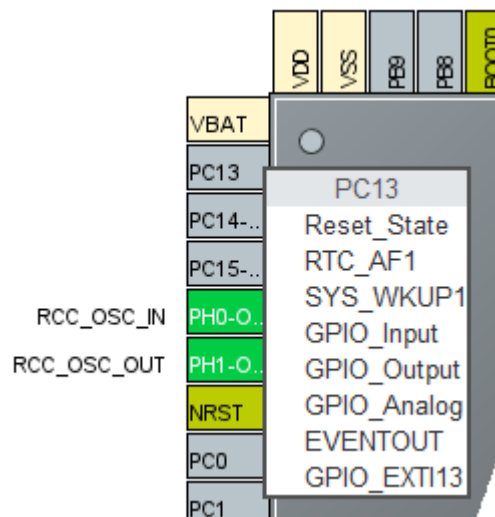
No primeiro bloco do código, as configurações da mensagem são configuradas, em especial o identificador, que permite identificar o conteúdo da mensagem pela tabela mostrada previamente. Em seguida, o vetor de dados é carregado com os dados. Como os dados possuem 12 bits de informação, é preciso dividir em 2 bytes para não haver perda de informação. O descolamento de 8 bits para esquerda faz com que os bits mais significativos sejam salvos, e a máscara com 0xff, faz com que apenas os 8 bits menos significativos sejam salvos. A operação inversa é feita posteriormente para recuperar o dado original.

Por fim, a função *HAL_CAN_AddTxMessage* irá receber todos os parâmetros necessários e se encarregar de enviar os dados.

3.2.4 Entradas e Saídas

A configuração de entradas e saídas digitais permite que o microcontrolador tenha uma interface com componentes elétricos da placa. Para configurar um pino como entrada ou saída, primeiramente é preciso abrir a interface gráfica do *STMCubeMX*, selecionar a porta desejada, e escolher a opção apropriada. A Figura 31 mostra esse passo:

Figura 31: Configuração da porta



Fonte: Autor

Ao clicar no pino PC13 como exemplo, as seguintes opções são mostradas. É importante notar que cada pino possui funcionalidades diferentes. *GPIO_Input* irá configurar o pino como uma entrada digital, enquanto *GPIO_Output* como saída. Em ambos os casos, também é possível configurar resistores de *pullup* e *pulldown* que se conectam internamente a porta.

Caso uma porta esteja configurada como saída, é possível alterar seu estado utilizando as funções implementadas. A principal, *HAL_GPIO_WritePin(GPIOx, GPIO_Pin, PinState)*, tem como argumentos uma porta e seu estado (1 ou 0) desejado. Assim, a função *HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, 1)* irá definir a porta PC13 em nível lógico alto por exemplo. Existe também a função *HAL_GPIO_TogglePin(GPIOx, GPIO_Pin)*, que inverte o estado atual de uma porta específica. Essa função é bastante utilizada para piscar os LEDs implementados.

Já para as portas configuradas como entrada, a função *HAL_GPIO_ReadPin(GPIOx, GPIO_Pin)* é usada, que retorna o nível lógico da porta especificada (1 ou 0). Porém, isso envolve o processador verificar continuamente o estado da porta, e em aplicações em que é necessário verificar se houve uma mudança de estado, o uso de uma interrupção externa é mais indicado. No caso da Figura 31, a opção *GPIO_EXTI13* seria selecionada. Nesse caso, também é possível escolher se a interrupção será detectada em borda de subida, de descida, ou em ambas. O Código 5 mostra um exemplo do módulo de controle, em que várias entradas estão configuradas como interrupção externa.

Código 5: Interrupção externa

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    //Troca de marcha
    if(GPIO_Pin == GPIO_PIN_4 && !HAL_GPIO_ReadPin(IN_UP) &&
    !contador_cooldown)
    {
        Troca_Marcha(UPSHIFT);
        contador_cooldown = COOLDOWN;
    }
    if(GPIO_Pin == GPIO_PIN_7 && !HAL_GPIO_ReadPin(IN_DW) &&
    !contador_cooldown)
    {
        Troca_Marcha(DOWNSHIFT);
        contador_cooldown = COOLDOWN;
    }

    //Troca de marcha Automatica
    if(GPIO_Pin == GPIO_PIN_3 && HAL_GPIO_ReadPin(IN_FT600) &&
    !contador_cooldown)
    {
        Troca_Marcha(UPSHIFT);
        contador_cooldown = COOLDOWN;
    }
    //Embreagem
    if(GPIO_Pin == GPIO_PIN_9 && !HAL_GPIO_ReadPin(IN_EMB))
    {
        Embreagem_Ligada();
    }
    if(GPIO_Pin == GPIO_PIN_9 && HAL_GPIO_ReadPin(IN_EMB))
    {
        Embreagem_Desligada();
    }
}
```

É importante notar que uma mesma interrupção é acionada por múltiplas entradas distintas, sendo assim necessário fazer a verificação de qual pino de fato foi responsável pela chamada da interrupção. Isso é feito pelo parâmetro *GPIO_Pin*, que passa o número do pino da porta acionada. Por medida de segurança, também é verificado se o estado da porta em si está

no estado esperado. Por fim, uma variável de *cooldown* não permite que uma mesma porta seja acionada mais de uma vez dentro de um período especificado, evitando assim que ruídos durante a mudança de estado possam, por exemplo, realizar duas trocas de marcha quando apenas uma foi requisitada.

3.2.5 Entradas Analógicas

As entradas analógicas do microcontrolador utilizado permitem a aquisição de até 16 sinais analógicos conectados nas entradas especificadas. Cada uma das entradas faz a leitura de uma tensão na faixa de 0V e 3,3V, possuindo uma resolução de 12 bits. Assim, um valor de 0 até 4095 será obtido, onde 0 significa 0V, e 4095 significa 3,3V (mais precisamente, a tensão de alimentação, que pode variar).

Dois fatores importantes na aquisição de sinais analógicos, são a resolução e frequência de aquisição. Uma resolução baixa não permite determinar com precisão a leitura do sensor, pois existe uma margem de erro muito grande. A resolução utilizada de 12 bits permite dividir os dados em 4096 possíveis posições, permitindo assim determinar com precisão a leitura do sensor no momento da aquisição.

Sobre a frequência de aquisição, uma ferramenta muito importante utilizada para a aumentar é o DMA, que permite que os valores das leituras sejam atualizados diretamente na memória do sistema independentemente do processador.

A configuração das entradas analógicas é relativamente simples, bastando habilitar as portas necessárias e algumas configurações básicas como resolução e modo de aquisição, no caso o DMA. Após essas configurações, para iniciar a leitura é preciso chamar uma função que leva como um dos parâmetros um vetor que será usado para guardar os dados na medida que eles forem capturados. É importante que o tamanho do vetor seja o mesmo das portas analógicas habilitadas. Assim, a posição 0 do vetor será sempre atualizada com a leitura da primeira porta, e assim por diante. Um exemplo de como utilizar esse vetor mencionado está presente no Código 4 já mostrado.

3.2.6 Comunicação Serial

Um dos protocolos de comunicação mais utilizado com microcontroladores é a comunicação serial, que tem uma implementação simples e permite que dois dispositivos troquem informações entre si. No protótipo, a comunicação serial é usada para enviar dados para o transmissor XBee e obter dados do GPS.

A comunicação serial utiliza dois pinos do microcontrolador: um que faz o envio dos dados (*TX*) e outro que faz o recebimento (*RX*). É importante o que o pino *TX* de um dispositivo se conecte no *RX* do outro dispositivo, e vice-versa. Ambos os dispositivos também precisam ser configurados com a mesma taxa de transmissão.

A implementação de uma comunicação serial no micro é bem simples, sendo primeiro necessário habilitar a interface e escolher os pinos utilizados (uma mesma interface pode ter mais de uma opção de pinos utilizados). Após isso, a configuração dos parâmetros é feita, como mostra a Figura 32.

Figura 32: Configuração da porta serial

▼ Basic Parameters	
Baud Rate	115200 Bits/s
Word Length	8 Bits (including Parity)
Parity	None
Stop Bits	1
▼ Advanced Parameters	
Data Direction	Receive and Transmit
Over Sampling	16 Samples

Fonte: Autor

Para realizar o envio de uma mensagem, é preciso primeiramente colocar os dados a serem enviados em um vetor, e em seguida utilizar a função *HAL_UART_Transmit*, que leva como parâmetros principais o vetor dos dados a serem enviados, e o tamanho da mensagem. O Código 6 mostra como o módulo transmissor enviar para o *XBee* as mensagens recebidas na rede CAN.

Código 6: Envio de dados na porta serial

```

void Escrever_Dados_Transmissor()
{
    mensagem[0] = 'U';
    mensagem[1] = 'S';
    mensagem[2] = 'P';
    mensagem[3] = RxMessage.ExtId & 0xFF;
    mensagem[4] = RxMessage.DLC;
    checksum = 0;
    for(int i = 0; i < mensagem[4]; i++)
    {
        mensagem[5+i] = rxData[i];
        checksum += rxData[i];
    }
    mensagem[5 + RxMessage.DLC] = checksum % 255;

    HAL_UART_Transmit(&huart1, mensagem, 6+RxMessage.DLC, 10);
}

```

Os três primeiros *bytes* servem como filtro de recebimento, pois qualquer mensagem recebida que não contenha esses bytes nessa sequência, será ignorada. Em seguida, o identificador da mensagem e seu tamanho são escritos. Na sequência os dados em si são guardados no vetor, e junto com isso, um sistema de verificação é feito, no qual os dados são somados e a operação de resto é aplicada. Esse sistema de verificação é uma das formas de *checksum* mais básicas que existem, mas é o suficiente para a aplicação. Por fim, a mensagem é enviada usando a função já mencionada. É importante notar que cada mensagem pode possuir um tamanho diferente.

3.2.7 Estrutura de execução

Um ponto muito importante na organização dos códigos é definir de maneira bem estruturada a chamada das funções do microcontrolador. Essas funções podem ser chamadas no início do código, dentro de alguma interrupção, e dentro da estrutura de repetição principal.

As funções responsáveis por inicialização de parâmetros e periféricos são chamadas no início da função *main*. O código de inicialização do módulo de aquisição pode ser visto no Código 7:

Código 7: Inicialização do programa

```

int main(void)
{
    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */
    HAL_Init();

    /* Configure the system clock */
    SystemClock_Config();

    /* Initialize all configured peripherals */
    MX_GPIO_Init();
    MX_DMA_Init();
    MX_ADC1_Init();
    MX_I2C1_Init();
    MX_SPI3_Init();
    MX_TIM1_Init();
    MX_CAN1_Init();

    /* USER CODE BEGIN 2 */
    HAL_TIM_Base_Start_IT(&htim1);
    Configura_CAN();
    HAL_CAN_ActivateNotification(&hcan1, CAN_IT_RX_FIFO0_MSG_PENDING);
    HAL_ADC_Start_DMA(&hadc1, ADC_BUFFER, 8);
    Inicializa_Datalogger();

```

Todas as chamadas até *MX_CAN1_Init* são geradas automaticamente pelo *STMCubeMX* e são responsáveis por inicializar o sistema. As funções abaixo são gerenciadas pelo usuário e em geral também são responsáveis por inicializar os periféricos. Nesse caso, essas funções são responsáveis por inicializar o *Timer 1*, configurar os filtros da CAN, ativar o recebimento das mensagens CAN, inicializar a aquisição dos sinais analógicos e iniciar o salvamento de dados no cartão SD.

As funções chamadas durante uma interrupção normalmente são executadas em resposta a uma ação (recebimento de mensagens, acionamento de algum botão, etc.). Assim, elas não possuem uma frequência fixa de execução, sendo executadas sempre que necessário. Em alguns casos, é preciso tomar cuidado para que essas chamadas não acontecem de maneira excessiva e ocupem todo o poder de processamento.

Por fim, temos a estrutura principal do código, a estrutura de repetição principal. Nesse bloco, são chamadas todas as funções que devem ser executadas em uma frequência fixa. O Código 8 mostra o exemplo do módulo de aquisição:

Código 8: Estrutura de repetição principal

```

while (1)
{
    if(contador_msg_1 >= 10)
    {
        Enviar_Mensagem_1();
        contador_msg_1 = 0;
    }
    if(contador_msg_2 >= 50)
    {
        Enviar_Mensagem_2();
        contador_msg_2 = 0;
    }
    if(contador_verify >= 200)
    {
        Verify_Sensores();
        Enviar_Mensagem_Verify();
        Envia_Mensagem_Transmissor();
        contador_verify = 0;
    }
}

```

Ao usar “1” na condição do *while*, o código ficará executando o laço indefinidamente até ser reiniciado. Dentro do *while* existem vários laços condicionais. Todos utilizam uma variável de contador, que é incrementada a cada 1ms como já mostrado nesse trabalho, e um valor que determina o período de execução. Pensando no primeiro laço como exemplo, ao iniciar o código o valor de *contador_msg_1* será de 0, e será incrementado a cada 10ms. Uma vez que o valor for maior ou igual que 10, a função *Enviar_Mensagem_1* é chamada, e o contador é reiniciado novamente. Assim, a função é executada a cada 10ms, ou a uma frequência de 100Hz. O motivo de se utilizar o operador maior ou igual, é de que uma vez que o código utiliza bastantes estruturas de interrupção, é provável que eventualmente o contador seja incrementado mais de uma vez antes do laço condicional poder verificar. Assim, se a condição fosse do contador ser igual a 10, e o valor fosse para 11, as funções dentro do laço não seriam mais executadas.

Utilizando essa estrutura, é possível executar funções diferentes independentemente uma das outras, com frequências diferentes. No caso acima, o primeiro laço é responsável por obter os dados de um sensor que necessita alta frequência de aquisição, logo é executado 100Hz. O laço seguinte obtém os dados dos sensores que não necessitam de uma frequência tão alta, sendo assim executado a 20Hz. Por fim, o último laço envia mensagens que servem para verificar o funcionamento do sistema, não precisando assim de uma frequência alta, sendo executado a 5Hz.

3.3 Sensores e Atuadores

Após ter o desenho das placas finalizadas e entender as ferramentas e recursos que podem ser implementados no *firmware*, é possível de fato criar a interface entre o sistema eletrônico e carro em si. Será visto aqui os pontos importantes a serem considerados na implementação dos sensores e atuadores do protótipo, com foco maior no *firmware* e em cuidados especiais.

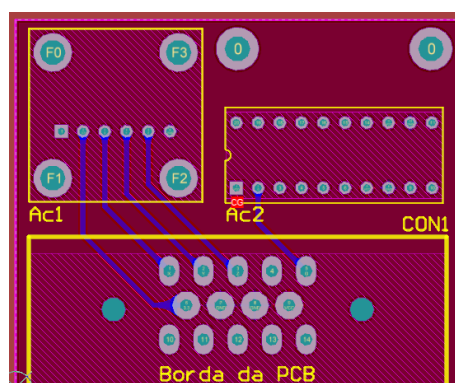
3.3.1 Sensores analógicos

Os sensores analógicos têm suas tensões lidas nas placas pelos conversores ADC. Eles possuem diversas aplicações e permitem um estudo aprofundado do veículo.

3.3.1.1 Acelerômetro e Giroscópio

Dois sensores muito importantes na análise da dinâmica do veículo são o acelerômetro e o giroscópio. O acelerômetro utilizado é o ADXL325Z produzido pela *Analog Devices*, que faz a medida da aceleração em 3 eixos, sendo que cada eixo é representado por uma saída analógica de 0V a 3,3V. O giroscópio usado é o ADXRS642, também produzido pela *Analog Devices*, que mede a velocidade angular em apenas um eixo. A saída analógica de 0V a 5V é proporcional a velocidade angular. Ambos estão localizados em uma placa de circuito impresso auxiliar, que é fixada perto do centro de gravidade do carro, o que é essencial para parte de análise da dinâmica veicular. Essa placa possui um conector automotivo conectado na placa de aquisição principal. Ela recebe as alimentações de 3,3V para o acelerômetro, 5V para o giroscópio, e envia os 4 sinais analógicos para serem lidos. A Figura 33 mostra o desenho desta placa, na qual *Ac1* representa o acelerômetro e *Ac2* o giroscópio:

Figura 33: PCI do acelerômetro e giroscópio



Fonte: Autor

Uma vez que o sinal é recebido na placa de aquisição, ele passa pelos filtros passa-baixa já vistos e são convertidos para valores digitais de 12 bits em uma frequência de 50Hz. Os dados são então enviados na rede CAN e gravados no cartão SD da própria placa.

O acelerômetro é calibrado com a placa fora do carro. Os valores são lidos com a placa em diferentes orientações, onde um eixo irá ter uma aceleração de 1G ($9,8 \text{ m/s}^2$) e os outros dois de 0. Com isso, é possível interpolar os pontos linearmente e obter as funções de transferência para os 3 eixos. A utilização desse e de outros sensores será vista no Capítulo 4.

3.3.1.2 Pressão de Freio

O sistema de freio do veículo possui duas linhas de fluídos, uma para os freios dianteiros, e outra para os traseiros. Utilizando sensores de pressão que são encaixados na linha, é possível determinar a pressão de ambas as linhas a todo instante, o que permite verificar rapidamente se há algum problema com alguma das linhas além de poder analisar o perfil de frenagem dos pilotos. A Figura 34 mostra o sensor utilizado, modelo 3PP6-5 da *Sensata Technologies*. O sensor é alimentado com 5V e possui uma saída analógica de 0V a 4.75V, que corresponde a uma escala de 0MPa a 30MPa.

Figura 34: Sensor de pressão de freio



Fonte: *Sensata*. 2020. Disponível em

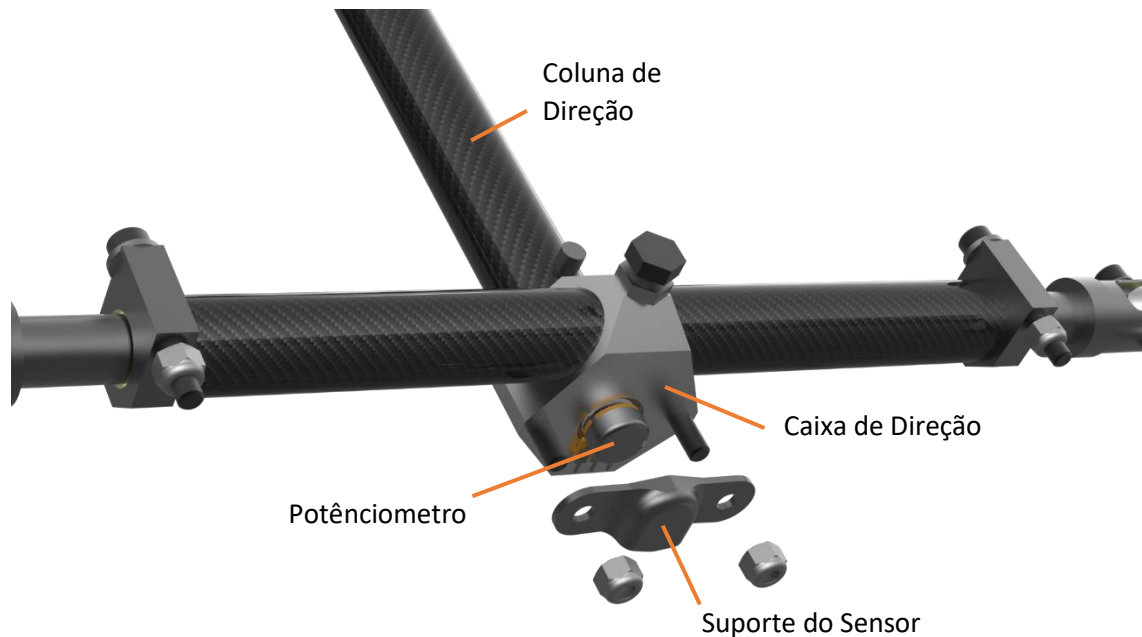
<https://www.sensata.com/products/pressure-sensors-switches/3pp8-hermetic-pressure-sensors>

3.3.1.3 Esterço

O sensor de esterço obtém o ângulo do volante de direção, e permite analisar como diferentes pilotos se comportam em curvas, além de observar o comportamento do carro. Para

isso, é utilizado um potenciômetro comum de $10k\Omega$ tipo B (comportamento linear). O potenciômetro é posicionado dentro da coluna de direção, de modo que a parte rotativa esteja em contato com o pinhão que está conectado na coluna de direção, que rotaciona, enquanto a base fica fixada na caixa de direção por um suporte. A Figura 35 mostra uma renderização que permite ver o posicionamento do sensor:

Figura 35: Posicionamento sensor de esterço



Fonte: Autor

É importante após instalar o sensor, fazer sua calibração. Isso é feito rotacionando o volante 90° para a esquerda, 90° para direita e fazendo a interpolação desses pontos com os valores lidos.

3.3.1.4 Curso de suspensão

Em carros de competição, é muito importante ter um ajuste de suspensão que aumente o máximo possível o contato entre os pneus e o asfalto, distribuindo a carga de maneira equilibrada entre as quatro rodas. Porém, sem o sensoriamento adequado, a única maneira de verificar se o ajuste está bem feito é pelos comentários dos pilotos. Os sensores de curso de suspensão são potenciômetros lineares que servem para capturar a posição dos amortecedores ao longo da volta, permitindo assim análises complementares que permitem entender melhor o comportamento do carro.

O sensor utilizado é o modelo LPPS-22 produzido pela *Harold G. Schaevitz Industries*. Por ser um potenciômetro, a tensão será 0V quando comprimido, e 5V (tensão de alimentação)

quando esticado, o que equivale a 75mm de deslocamento. O sensor e seu posicionamento no protótipo podem ser vistos nas Figura 36 e Figura 37. O sensor é posicionado em paralelo com o conjunto amortecedor/mola, permitindo assim medir a posições do amortecedor.

Figura 36: Sensor de curso da suspensão



Fonte: *Harold G. Schaevitz Industries*. 2020. Disponível em <https://www.hgsind.com/product/lpps-22-linear-potentiometer-position-sensor-rod-end-joints>

Figura 37: Posicionamento sensor de curso da suspensão no carro



Fonte: Autor

O modelo escolhido permite um deslocamento de 75mm, o que é suficiente para a geometria de suspensão usada. O sensor não precisa ser calibrado, uma vez que uma tensão de 0V significa um deslocamento de 0mm, e quando a tensão for igual a tensão de referência, o deslocamento é de 75mm. Porém, para cada arquivo de dados gerado, um *offset* é aplicado fazendo com que os valores sejam 0 com o carro parado, o que facilita a análise e permite o cálculo de alguns canais matemáticos como o ângulo de rolagem, o que será visto mais para frente.

3.3.2 Sistema de Posicionamento Global (GPS)

A utilização de um sistema de GPS no carro permite salvar a posição e velocidade do veículo nos arquivos de dados, o que facilita a análise de dados, permitindo identificar facilmente em qual parte da pista o carro estava em qualquer dado momento. O módulo utilizado é um *u-blox NEO-6M*, que se comunica com o microcontrolador utilizando uma porta serial. O dispositivo envia vários dados de maneira predeterminada na porta serial, e um código foi implementado para fazer a decodificação dos dados.

A Figura 38 mostra o programa de análise de dados colocando os pontos obtidos pelo GPS por cima de um mapa de satélite e mostra o funcionamento do sistema. Com esse mesmo programa, é possível criar um mapa da pista que mostra a posição do protótipo no momento selecionado.

Figura 38: Traçado do carro gerado pelo GPS

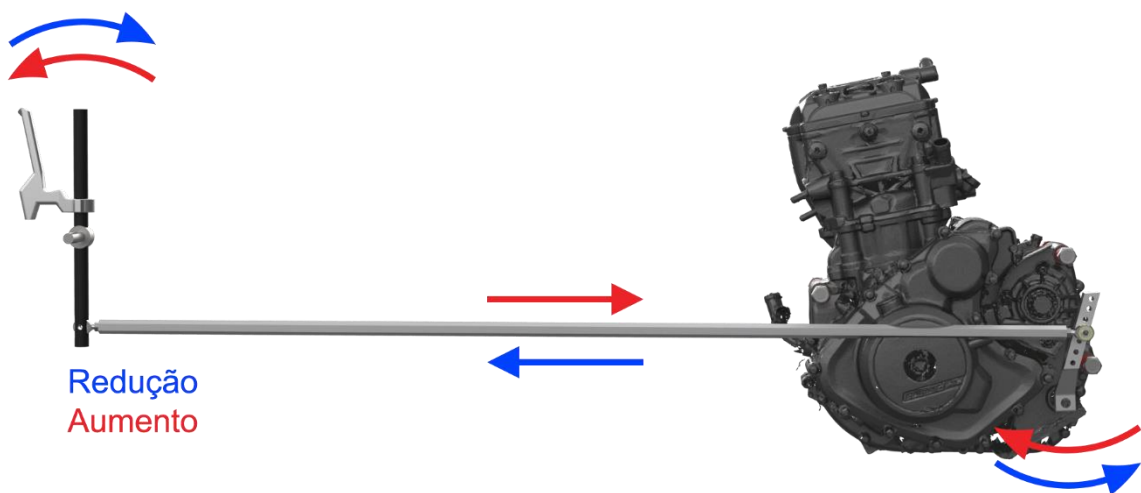


Fonte: Autor

3.3.3 Atuadores Pneumáticos

Uma das ações que o piloto mais realiza é a troca de marchas. O motor utilizado possui um câmbio sequencial, que permite o aumento de marchas com um único movimento e a redução com o movimento contrário. A Figura 39 ilustra esse mecanismo, sendo que os movimentos em azul indicam o movimento de redução, e os vermelhos do aumento de marcha. A alavanca no qual o piloto interage (na esquerda da figura) está conectada diretamente ao trambulador por meio de um varão de alumínio.

Figura 39: Sistema manual de troca de marchas



Fonte: Autor

O principal problema desse sistema, é de necessitar que o piloto retire uma das mãos do volante para realizar as trocas. Além disso, embora a força necessária para realizar a troca não seja muito alta, o movimento total necessário é considerável, e considerando que uma volta costuma ter entre 15 a 20 trocas, a fadiga no braço do piloto se torna um fator considerável no desempenho total.

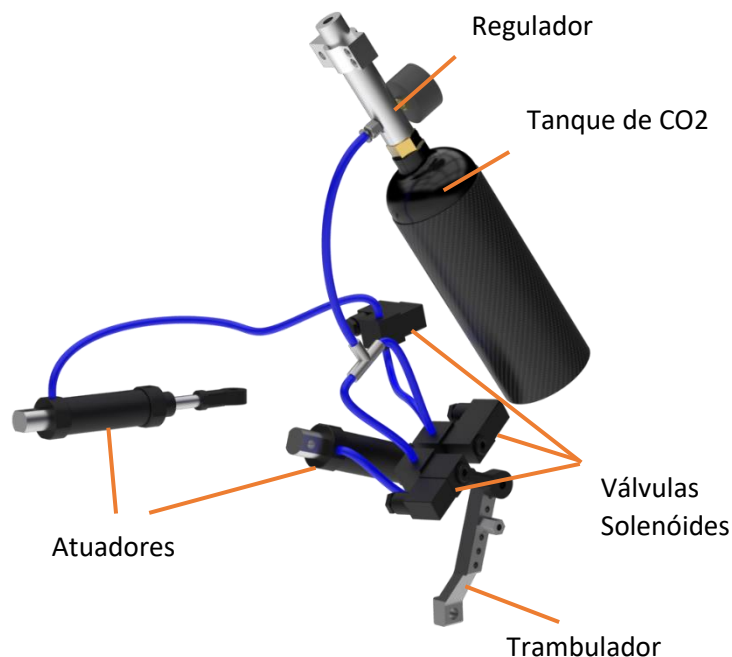
Dessa maneira, um sistema pneumático é utilizado para realizar as trocas de marcha apenas com botões, conhecidos como borboletas, atrás do volante. Essas borboletas enviam um sinal para o módulo de controle que gerencia o sistema pneumático, fazendo com que um atuador rotacione o trambulador, realizando assim a troca. Os principais componentes do sistema são:

- Tanque de CO₂: Armazena gás CO₂ pressurizado que funciona como a alimentação do sistema;

- Regulador de pressão: Controla a pressão do sistema, mantendo uma pressão relativamente constante independente da pressão de entrada e fluxo na saída;
- Linhas de ar: Mangueiras que fazem as conexões entre os componentes;
- Válvulas solenoides: controlam o fluxo de ar para determinadas regiões utilizando sinais elétricos, que fecham ou abrem as válvulas;
- Atuadores pneumáticos: dispositivo que convertem o gás pressurizado em movimento linear.

Membros do subsistema do motor são responsáveis por implementar todo o sistema mecânico/pneumático. Esse sistema utilizado usa dois atuadores pneumáticos, sendo um para a troca de marchas, e o segundo para o acionamento da embreagem. A Figura 40 mostra o sistema completo.

Figura 40: Sistema pneumático



Fonte: Autor

O acionamento eletrônico do sistema é feito pelo acionamento das válvulas solenoides. Porém a interface com o piloto é feita através do volante, que possui 2 borboletas para troca de marchas e 4 botões, sendo um deles para o acionamento da embreagem. A Figura 41 mostra o volante utilizado:

Figura 41: Volante do protótipo



Fonte: Autor

Analisando primeiramente o acionamento da embreagem, todo momento que o botão é apertado ou solto, uma interrupção é acionada no módulo de controle, que verifica o estado do botão e aciona ou desaciona a embreagem. Nesse caso, enquanto o piloto segurar o botão, a embreagem estará acionada. Uma variável armazena o estado de acionamento, que é salvo juntamente com os outros dados da aquisição de dados. O Código 9 demonstra esse funcionamento. Já o Código 10 mostra as funções que habilitam e desabilitam a válvula da embreagem.

Código 9: Interrupção do botão da embreagem

```
//Acionamento embreagem dentro da interrupção
if(GPIO_Pin == GPIO_PIN_9 && !HAL_GPIO_ReadPin(IN_EMB))
{
    Embreagem_Ligada();
}
if(GPIO_Pin == GPIO_PIN_9 && HAL_GPIO_ReadPin(IN_EMB))
{
    Embreagem_Desligada();
}
```

Código 10: Acionamento do atuador da embreagem

```
void Embreagem_Ligada()
{
    HAL_GPIO_WritePin(OUT_EMB, ATUADOR_ON);
    embreagem = 1;
}

void Embreagem_Desligada()
{
    HAL_GPIO_WritePin(OUT_EMB, ATUADOR_OFF);
    embreagem = 0;
}
```

O acionamento da embreagem não envolve quase nenhuma lógica de acionamento. O sistema de troca de marcha, porém é mais complexo e requer a calibração de alguns parâmetros. Primeiramente, o Código 11 mostra as interrupções relacionadas a troca de marcha.

Código 11: Interrupção das trocas de marcha

```
//Troca de marcha
if(GPIO_Pin == GPIO_PIN_4 && !HAL_GPIO_ReadPin(IN_UP) && !contador_cooldown)
{
    Troca_Marcha(UPSHIFT);
    contador_cooldown = COOLDOWN;
}
if(GPIO_Pin == GPIO_PIN_7 && !HAL_GPIO_ReadPin(IN_DW) && !contador_cooldown)
{
    Troca_Marcha(DOWNSHIFT);
    contador_cooldown = COOLDOWN;
}

//Troca de marcha Automatica
if(GPIO_Pin == GPIO_PIN_3 && HAL_GPIO_ReadPin(IN_FT600) && !contador_cooldown)
{
    Troca_Marcha(UPSHIFT);
    contador_cooldown = COOLDOWN;
}
```

Os dois primeiros laços estão relacionados as borboletas de aumento e redução, respectivamente. Já o terceiro laço é acionado quando a injeção envia um sinal para realizar a troca. Esse modo é usado nas provas de aceleração e permite que a injeção eletrônica faça as trocas de marcha no momento ideal. Isso, juntamente com o acionamento da embreagem por um sistema pneumático, elimina quase todo erro humano durante as provas de aceleração, permitindo tempos mais consistentes e ajustes mais precisos.

Partindo então para as estratégias de troca, o Código 12 mostra as ações realizadas nas trocas:

Código 12: Acionamento do atuador de troca de marchas

```

void Troca_Marcha(uint8_t troca)
{
    if(troca == UPSHIFT && contador_cooldown == 0)
    {
        //Aciona o atuador de troca
        HAL_GPIO_WritePin(OUT_UP, ATUADOR_ON);
        HAL_Delay(5);

        //Envia sinal para corte de ignição
        HAL_GPIO_WritePin(OUT_ECU, 1);
        HAL_Delay(20);

        //Sinal de corte não é mais necessário
        HAL_GPIO_WritePin(OUT_ECU, 0);
        HAL_Delay(tempo_upshift - 25);

        //Desliga o atuador
        HAL_GPIO_WritePin(OUT_UP, ATUADOR_OFF);

        num_upshift++;
    }
    if(troca == DOWNSHIFT && contador_cooldown == 0)
    {
        //Aciona o atuador de troca
        HAL_GPIO_WritePin(OUT_DW, ATUADOR_ON);
        HAL_Delay(tempo_downshift);

        //Desliga o atuador
        HAL_GPIO_WritePin(OUT_DW, ATUADOR_OFF);

        num_downshift++;
    }
}

```

Analisando inicialmente a redução de marcha, o módulo aciona a válvula por um tempo dado pela variável *tempo_downshift* e em seguida fecha a válvula. Por fim uma variável guarda o número de reduções total para salvamento. O tempo no qual o atuador fica ativado precisa ser calibrado cuidadosamente, pois um tempo muito curto não permitirá que a troca possa ser completa, enquanto um tempo muito alto pode causar danos ao câmbio. A estratégia de aumento de marcha é parecida, porém existe um sinal enviado a injeção eletrônica responsável por fazer um corte na ignição do motor por um curto período. Esse corte tem como objetivo reduzir o torque gerado do motor, aliviando assim a tensão no câmbio e permitindo que as engrenagens consigam desacoplar. O tempo do corte não é definido pelo código acima, e sim por uma configuração da injeção eletrônica. O sinal do corte só é enviado após 5ms para permitir que quando o corte ocorra, o atuador já esteja pressurizado e a troca ocorra mais rapidamente.

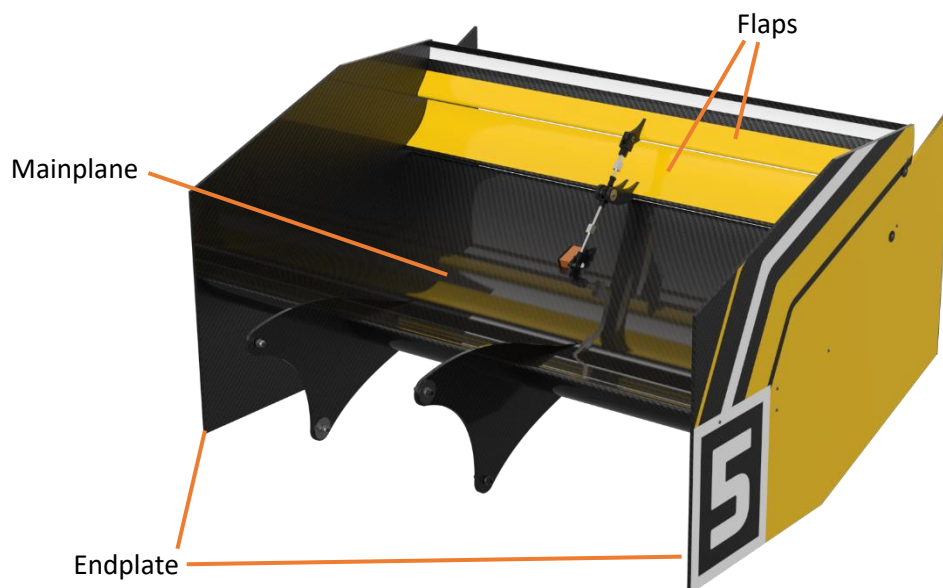
3.3.4 Sistema de Redução de Arrasto

Baseado no trabalho (BARBOSA PORTO, 2016), foi implementado um sistema de redução de arrasto que permite alterar o ângulo da asa a partir de um botão no volante, juntamente com os membros subsistema da aerodinâmica.

Os componentes aerodinâmicos do carro têm como principal função gerar *downforce*, e isso é alcançado ao direcionar o ar para cima do carro, criando uma força de reação que “empurra” o carro no asfalto, aumentando sua aderência total. Porém, quanto mais *downforce* é gerado, mais *drag*, ou arrasto, é gerado, o que aumenta a dificuldade do carro em acelerar. Assim, *downforce* é desejado nas curvas por permitir o carro contornar a uma velocidade maior, mas um baixo coeficiente de arrasto é desejado nas retas. Pensando nisso, o sistema de DRS permite mudar o nível de *downforce*, e conseqüentemente o arrasto, em momentos diferentes da pista. Para entender como o sistema funciona, primeiro é preciso entender os componentes da asa traseira.

A asa traseira do veículo possui 3 partes importantes: os *endplates* que são as paredes laterais, o *mainplane* que é a estrutura principal da asa e principal fonte de *downforce*, e os *flaps*, que aumentam a quantidade de *downforce* gerada, e permitem um ajuste nos ângulos de ataque como controle do nível de *downforce* criado. A Figura 42 mostra a asa traseira utilizada.

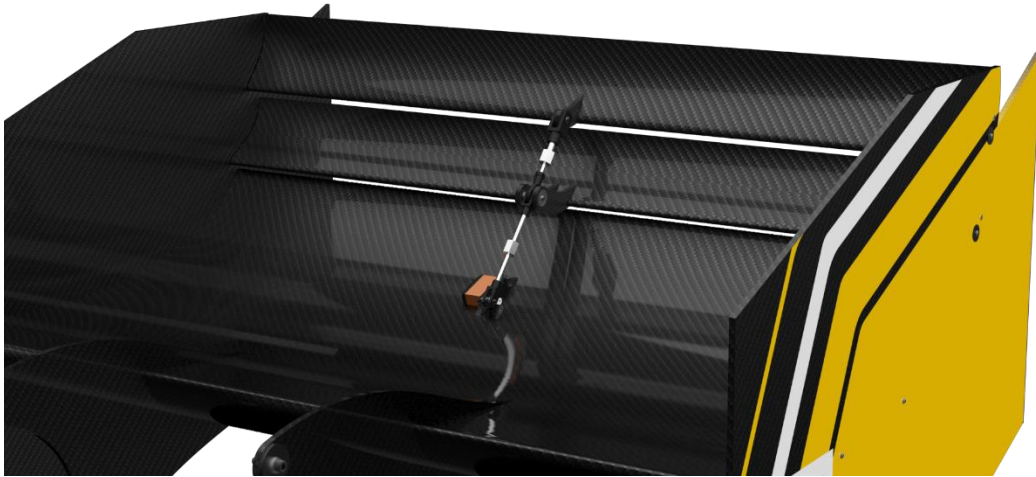
Figura 42: Componentes asa traseira



Fonte: Autor

Foi acoplado no *mainplane* um servomotor que se conecta aos dois *flaps* e permite ajustar o ângulo deles. As Figura 43 e Figura 44 mostram os *flaps* fechados e abertos pelo servo.

Figura 43: Asa traseira com *flaps* fechados



Fonte: Autor

Figura 44: Asa traseira com flaps abertos



Fonte: Autor

O controle do ângulo é feito pela rotação do eixo do servo. O sistema possui batentes de ângulo superiores e inferiores, nos quais o servomotor não irá mais conseguir rotacionar os ângulos corretamente e irá forçar o sistema que ficará travado. Por isso, é importante configurar os valores de ângulo do servomotor corretamente.

Esse controle, é feito através de um pulso de largura modular gerado pelo microcontrolador do módulo de controle. O ângulo do servomotor será proporcional à largura de pulso. Para poder gerar um pulso modular sem utilizar muito poder de processamento do

micro, o *TIM3* é utilizado no modo PWM. Uma vez configurado corretamente, o processador precisa apenas atualizar o valor da largura de pulso desejado.

A configuração do pulso modular é parecida com a do *Timer* visto anteriormente no item 3.2.2, porém existe um parâmetro adicional chamado de comparador. O parâmetro *Counter* determina um contador interno que é incrementado até o número configurado, momento no qual o *Timer* reinicia a contagem e o contador é zerado. Esse contador é comparado com o parâmetro comparador, e caso for menor o pulso fica em nível lógico alto, e se o contador for maior que o comparador, a saída fica em nível lógico baixo. Dessa maneira, para mudar a largura do pulso, basta mudar esse comparador. O Código 13 mostra como isso é feito:

Código 13: Controle do acionamento DRS

```
uint16_t
    angulo_off = 1126,
    angulo_on = 1170;
uint8_t
    posicao_servo = 0;

void Inicializa_DRS()
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, angulo_off);
    HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_3);
}

void DRS_Aberto()
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, angulo_on);
    posicao_servo = 1;
}

void DRS_Fechado()
{
    __HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_3, angulo_off+1);
    posicao_servo = 0;
}
```

A variável *posicao_servo* serve para gravar nos dados o estado do *DRS*, enquanto *angulo_off* e *angulo_on* armazenam os valores que definem o ângulo do servo. Esses valores são definidos com testes práticos para definir a melhor posição possível. A função de inicialização é chamada no início do código e aciona o servo na posição fechada. As funções que abrem e fecham o sistema utilizam a função `__HAL_TIM_SET_COMPARE` que muda o valor do registrador do comparador, permitindo assim mudar a largura do pulso. No exemplo do sistema fechada, o *duty cycle* é de $1126/1272 = 88,5\%$, porém é importante notar que o circuito de saída utiliza um optoisolador que acaba invertendo a saída. Assim, o *duty cycle* é de $11,5\%$, e a largura dos pulsos é de aproximadamente 2,3ms, considerando a frequência utilizada de 50Hz.

4 ANÁLISE DE DADOS

Esse capítulo irá mostrar algumas das técnicas utilizadas para analisar os dados coletados. A análise é dividida em duas partes: telemetria em tempo real e arquivos de dados salvos no cartão SD.

4.1 Telemetria

A utilização da telemetria permite a equipe monitorar os sinais vitais do carro na pista. Assim, é possível verificar se os sistemas monitorados estão funcionando de maneira correta, além de parar o carro no evento de um problema crítico (como por exemplo altas temperaturas no motor). Para entender o funcionamento da telemetria, primeiramente será visto como os dados são enviados do carro para um computador, e em seguida será mostrado o *software* que faz o processamento dos dados.

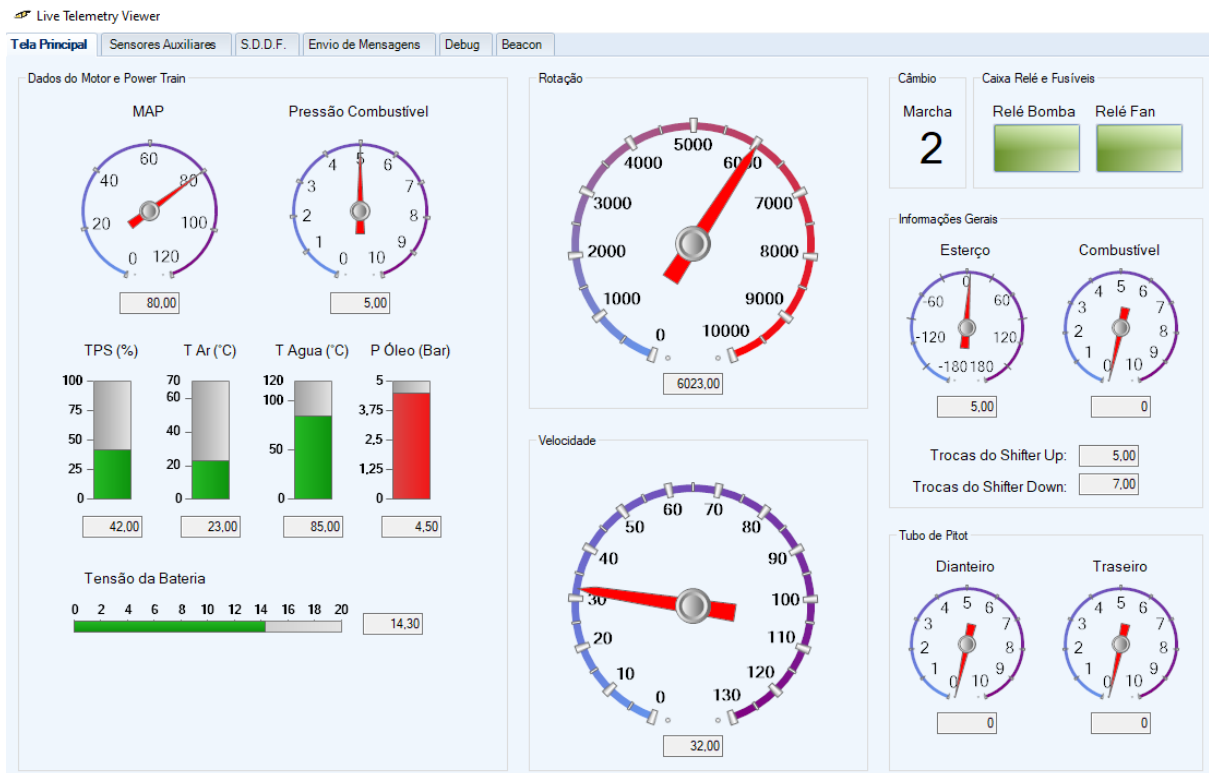
4.1.1 Transmissão dos dados

Para fazer a transmissão dos dados entre o carro e um computador, é utilizado um par de *XBee 900HP Pro*, como já mostrado anteriormente. Esses módulos de conectividade permitem o envio de dados por radiofrequência, utilizando uma frequência de 900MHz, o que permite uma distância de alcance suficientemente alta para o uso. Os dados são enviados e recebidos usando o protocolo serial. No caso do módulo transmissor localizado no carro, todos os dados da rede CAN são enviados na porta serial do módulo, enquanto no computador uma placa de interface é usada, que utiliza um conversor serial para USB (chip *FT232RL*).

4.1.2 Recebimento dos dados

O recebimento e processamento dos dados é feito utilizando um *software* desenvolvido na plataforma *Visual Studio*, utilizando a linguagem *C#*, que obtém os dados da porta serial, processa esses dados, e os mostra em várias telas diferentes, permitindo visualizar intuitivamente os dados mais importantes. A Figura 45 mostra a tela inicial desse programa.

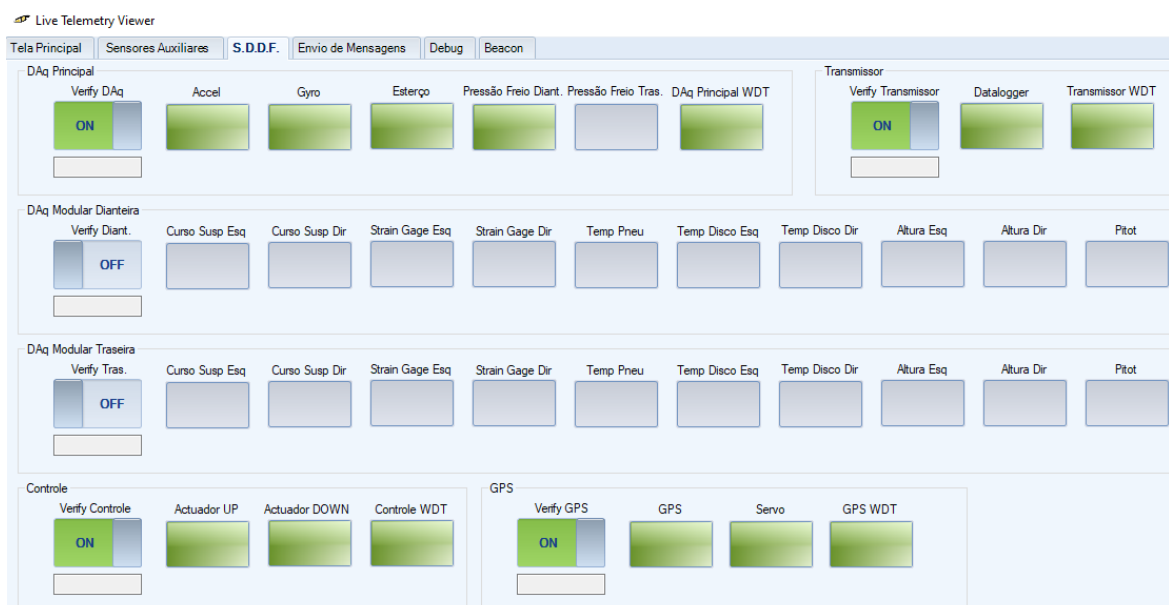
Figura 45: Programa utilizado para mostrar dados da telemetria



Fonte: Autor

O programa utilizado também possui um sistema de detecção de falhas, no qual mostra ao usuário rapidamente os módulos e sensores que estão funcionando corretamente, a fim de possibilitar um diagnóstico mais rápido e simples. Um exemplo dessa tela pode ser visto na Figura 46, onde o sensor de pressão de freio traseiro não estava conectado.

Figura 46: Sistema de detecção de falhas



Fonte: Autor

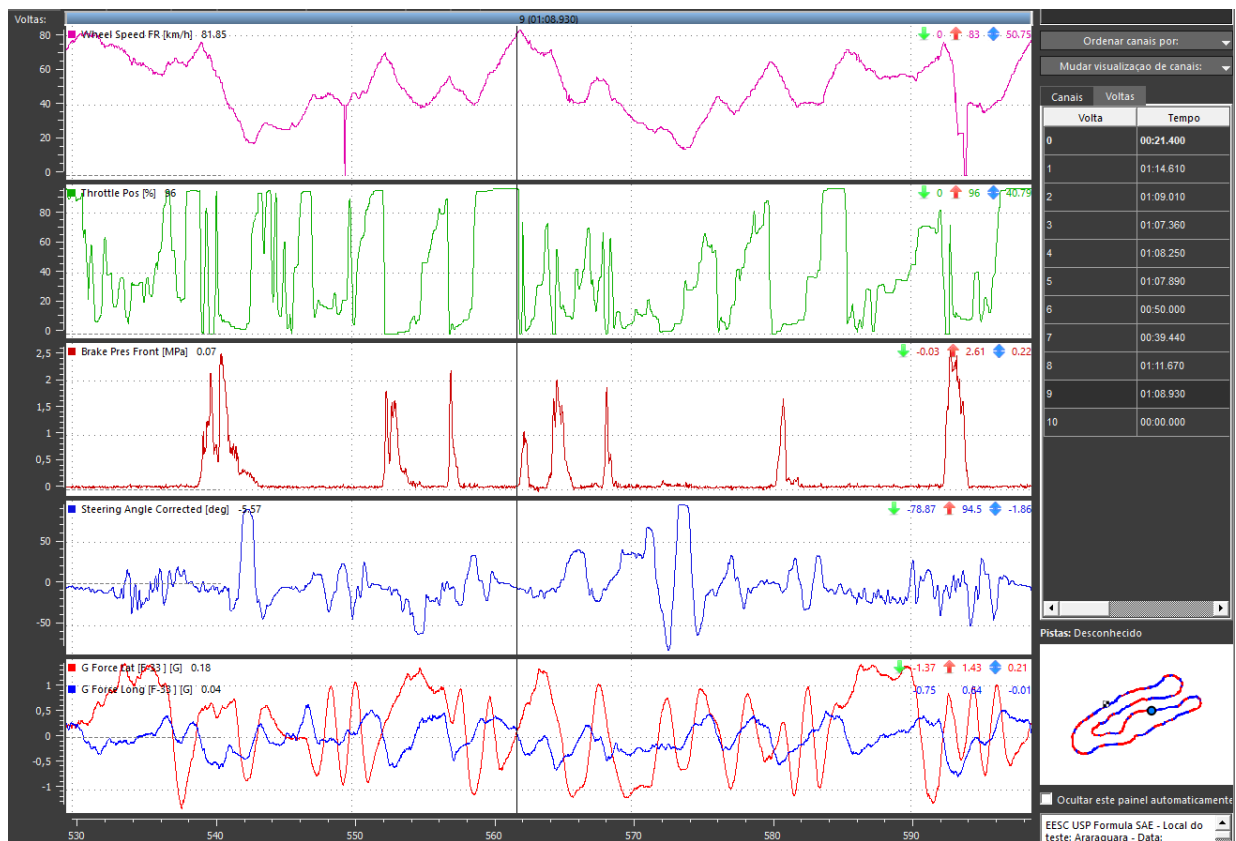
O programa também possui uma tela de depuração, na qual é possível ver todas as mensagens recebidas antes de serem processadas, e permite o salvamento dos dados recebidos na telemetria em um arquivo para análises posteriores caso necessário. Porém, na maioria das vezes, essa análise é feita utilizando o gravador de dados embarcado no carro.

4.2 Dados armazenados no carro

A maioria das análises de dados feitas após o carro ter andado são feitas utilizando o gravador de dados embarcado no carro. O módulo de aquisição grava os dados capturados e recebidos na rede CAN em um cartão SD. O mesmo programa responsável pelo recebimento da telemetria também faz o processamento desses dados e gera um arquivo de dados pronto para ser analisado, que contém vários canais de dados ao longo do tempo.

Após descarregar os dados e gerar o arquivo final, um *software* de análise de dados é utilizado. Nesse *software*, é possível adicionar vários gráficos, tabelas, e outras funções diferentes, como uma pista mostrando a posição do veículo. A Figura 47 mostra um exemplo de configuração usado.

Figura 47: Programa usado para analisar dados



Fonte: Autor

Nesse exemplo, um gráfico no tempo é utilizado, mostrando os canais de velocidade de roda, posição do acelerador, pressão de freio dianteira, esterço e as acelerações do carro. Esses dados são os principais para se analisar pilotos, pois estão relacionados a velocidade do carro e aos comandos realizados pelo piloto. No canto direito da imagem, também temos a numeração das voltas com o tempo de cada uma. Essas voltas são geradas utilizando o GPS, em que o ponto de largada é selecionado, e as voltas são geradas automaticamente. No canto inferior direito, uma pista do GPS é gerada que permite mostrar a posição do carro na pista no momento selecionado pelo cursor. A cor vermelha representa trechos de ganho de velocidade e da cor azul trechos de diminuição na velocidade.

A análise dos dados é dividida em três conjuntos: análise de pilotagem, que utiliza gráficos no tempo ou na distância; análise do comportamento do carro, que utiliza gráficos de dispersão, histogramas, e outras ferramentas para mostrar de maneira intuitiva como o protótipo estava se comportando nas condições do teste; e por fim, o uso de indicadores de desempenho, que resumem vários parâmetros obtidos em valores únicos para cada volta, afim de tentar perceber tendências na evolução do comportamento do veículo e dos pilotos.

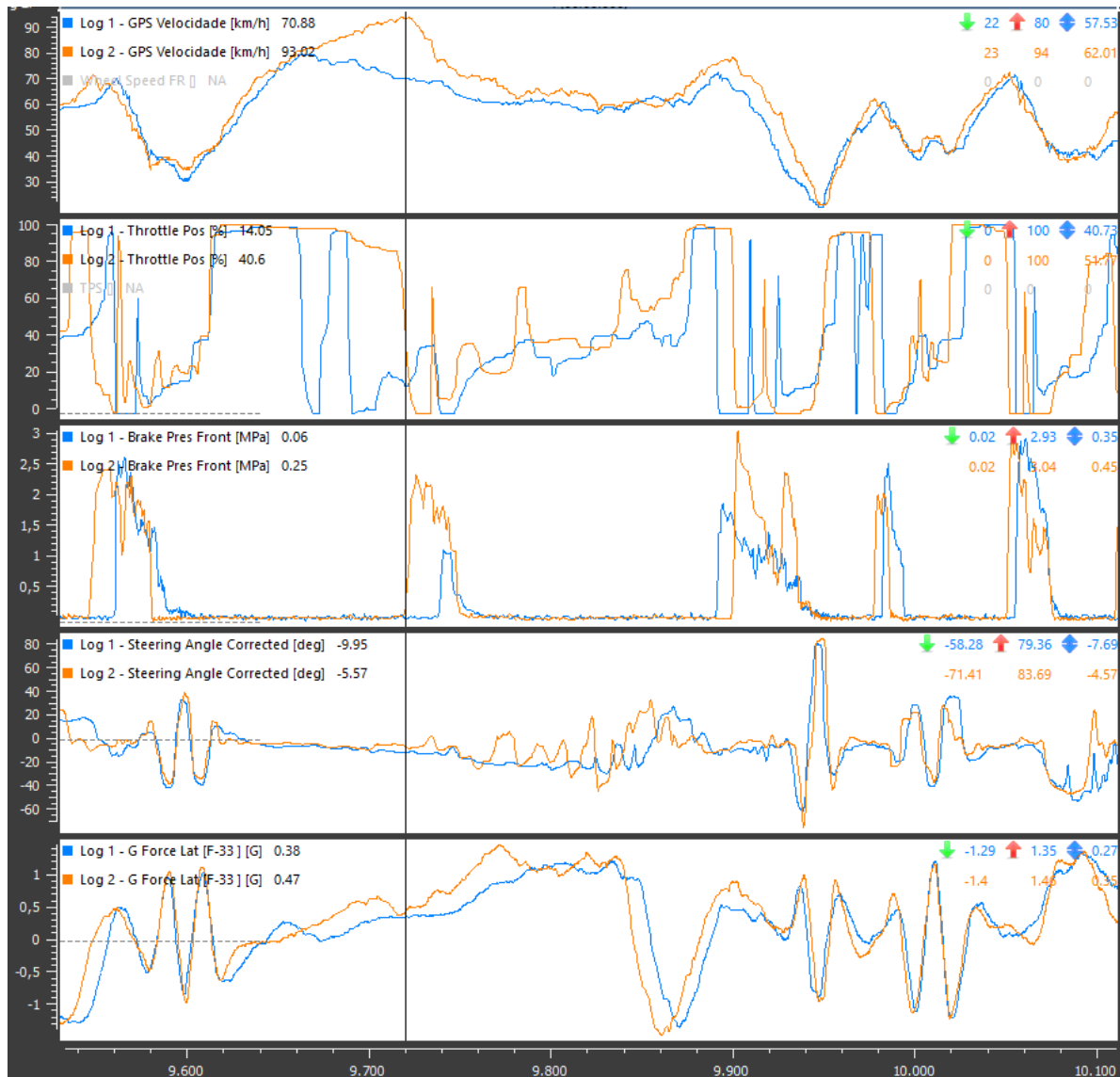
4.2.1 Análise de pilotagem

Em todas as competições automobilísticas, o piloto sempre possui um papel muito importante, pois o desempenho da equipe depende apenas dele enquanto o carro estiver na pista. Principalmente em competições universitárias, onde os pilotos em geral são amadores, é de extrema importância melhorar o máximo possível o nível de pilotagem, e a análise de dados tem um papel importante nessa função.

A principal forma de analisar a pilotagem é comparando com outro piloto. Nesse caso, é possível sobrepor os dados e comparar diretamente ações como frenagens e acelerações. Porém, se os pilotos tiverem um tempo de volta diferente, ao sobrepor os gráficos pelo tempo, ao longo da volta eles iram perder sincronia, pois um piloto está percorrendo a pista mais rapidamente que o outro. Para isso, é gerado um canal de distância, que faz uma integração do canal de velocidade. Os gráficos são então plotados em relação a distância na pista. Assim, independentemente dos tempos de volta, um mesmo ponto nos dados representa um mesmo local na pista. Dessa maneira, é fácil e intuitivo comparar pontos de frenagens e aplicação do acelerador, além de entender em quais pontos um piloto é mais rápido que o outro.

A Figura 48 mostra uma comparação em um setor da pista entre dois pilotos diferentes. É importante notar que no eixo vertical se encontra a distância na pista em metros.

Figura 48: Comparação entre pilotos



Fonte: Autor

Nesse exemplo da figura acima, temos o piloto laranja e o azul percorrendo um setor da pista. Temos as curvas de velocidade, pedal do acelerador, pressão de freio, esterço, e aceleração lateral. A análise começa pelo perfil de velocidade. É possível claramente ver que existe uma grande diferença de velocidade na posição do cursor (linha preta vertical), o que acabou prejudicando bastante o tempo do piloto azul. Analisando o segundo gráfico que mostra o pedal do acelerador, é possível ver que o piloto laranja mantém o pedal do acelerador apertado até o momento do cursor, enquanto o piloto azul solta por um breve momento, acelera por um curto período e solta novamente, tudo bem antes do piloto laranja, o que fez com que sua velocidade caísse enquanto a do laranja subisse. Logo após o cursor, ambos pilotos fazem uma frenagem. Analisando o terceiro gráfico, o piloto laranja freia mais cedo e com muito mais intensidade, gerando uma desaceleração maior. Em condições normais, isso seria ruim, porém,

como sua velocidade estava muito maior, é preciso realizar uma frenagem maior para alcançar a velocidade necessária para realizar a curva. Analisando o sinal do esterço após essa frenagem mencionada, é possível ver que o piloto laranja oscila o volante muito mais. Normalmente, isso significa que o piloto está mais perto do limite do carro, que acaba ficando mais instável sendo assim necessário realizar essas correções. Como o piloto azul estava com uma velocidade menor, ele não necessitou realizar essas correções. O último gráfico, de aceleração lateral, serve para mostrar em quais curvas um piloto consegue extrair mais do protótipo. É possível ver que no início e fim do setor, ambos pilotos possuem um perfil de aceleração lateral parecido, porém no trecho do meio, o piloto laranja consegue alcançar acelerações mais altas, devido sua maior velocidade de contorno. Nesse caso, a aceleração também serve para mostrar a necessidade de fazer a comparação por distância, pois mesmo com tempos de volta bastante diferentes, é possível ver que as curvas continuam sincronizadas.

A análise de pilotagem utilizando o método de comparação visto acima é bastante complexa e possui várias possibilidades. Os principais fatores a serem analisados são velocidade, que indica qual piloto está ganhando tempo em qual lugar da pista, acelerador, que indica o quão confiante o piloto está no carro e consegue acelerar e freio, no qual é analisado o local de aplicação, pressão máxima e perfil da curva. Um perfil desejado de frenagem é um no qual o piloto alcança a pressão máxima rapidamente e ao longo da frenagem vai soltando de maneira lenta e suave.

Uma outra maneira eficiente de analisar a pilotagem, é de sobrepor os dados obtidos por cima de um vídeo *onboard*. A Figura 49 mostra um quadro de um vídeo gerado que consta com uma sobreposição dos dados, permitindo analisar de maneira visual o desempenho do piloto.

Figura 49: Sobreposição de dados em vídeo



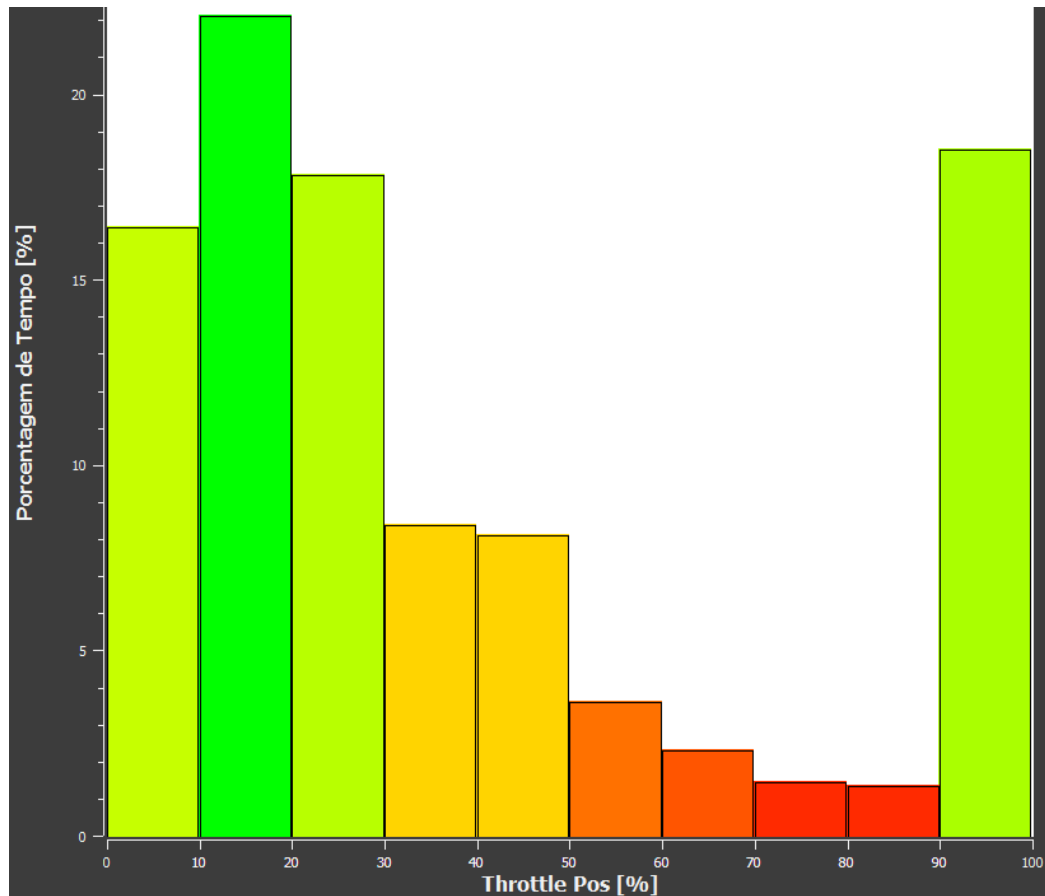
Fonte: Autor

4.2.2 Análise do comportamento do veículo

Diferentemente das análises de pilotagem, a análise do comportamento do veículo tem como objetivo achar tendências e resumir dados de maneira inteligente afim de entender melhor o comportamento dinâmico do carro. Para essas análises, os gráficos dos canais por tempo são menos usados uma vez que não representam corretamente o carro durante o período de teste completo. Por isso, ferramentas como diagramas de dispersão e histogramas são utilizados bastante. Essas análises são realizadas em conjunto com os membros dos subsistemas relacionados.

A primeira análise mostrada abaixo é bem simples, porém importante. A Figura 50 mostra um histograma da posição do acelerador durante tomadas de volta rápida. Com esse histograma, é possível observar que o pedal fica por um alto período fechado ou aberto até 30%, o que representa as curvas do circuito. O trecho intermediário, entre 30% e 90%, representa curvas de alta velocidade ou saídas de curva. Já o último trecho, de 90% para cima, representa os trechos de aceleração. Esse último trecho, indica que o carro consegue utilizar toda a potência do motor de maneira eficiente, pois o piloto quando necessita de potência, consegue abrir o acelerador por completo sem perder o controle do carro.

Figura 50: Histograma posição do acelerador



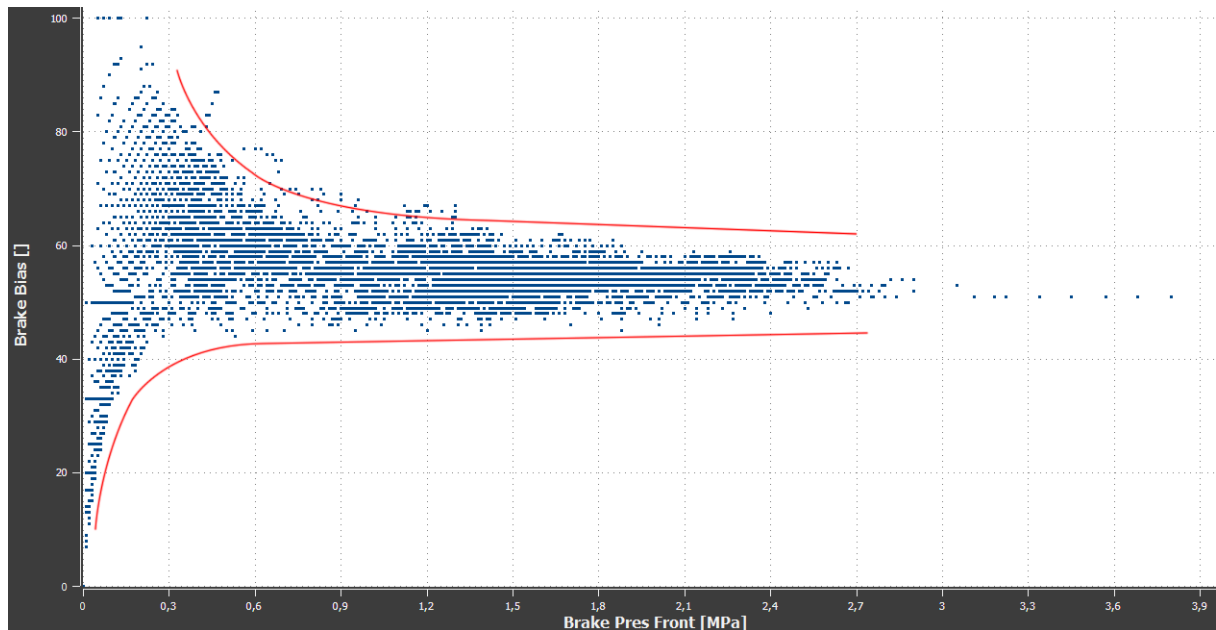
Fonte: Autor

O motor utilizado pela equipe é monocilíndrico, o que implica em um motor leve de baixa potência. A principal alternativa, um motor 4 cilindros, possui mais potência, mas é mais pesado. Um histograma de motor 4 cilindros nesse caso iria ter trechos intermediários muito maiores, pois o carro não consegue utilizar toda a potência disponível do motor. Assim, no nosso caso, é possível afirmar que conseguimos utilizar toda a potência do nosso motor, mas o carro também provavelmente tem condições de utilizar mais potência caso tivesse disponível.

Um outro tipo de gráfico bem útil no estudo do veículo é o gráfico de dispersão, que relaciona um dado diretamente com outro, permitindo perceber relações no carro. A Figura 51 mostra um diagrama de dispersão relacionando a distribuição de freio com a pressão de freio dianteira. A distribuição de freio é calculada usando a Equação 4.

$$Brake Bias [\%] = 100 * \frac{Brake Press Front}{Brake Press Front + Brake Press Rear} \quad (4)$$

Figura 51: Distribuição de freio por pressão de freio dianteira



Fonte: Autor

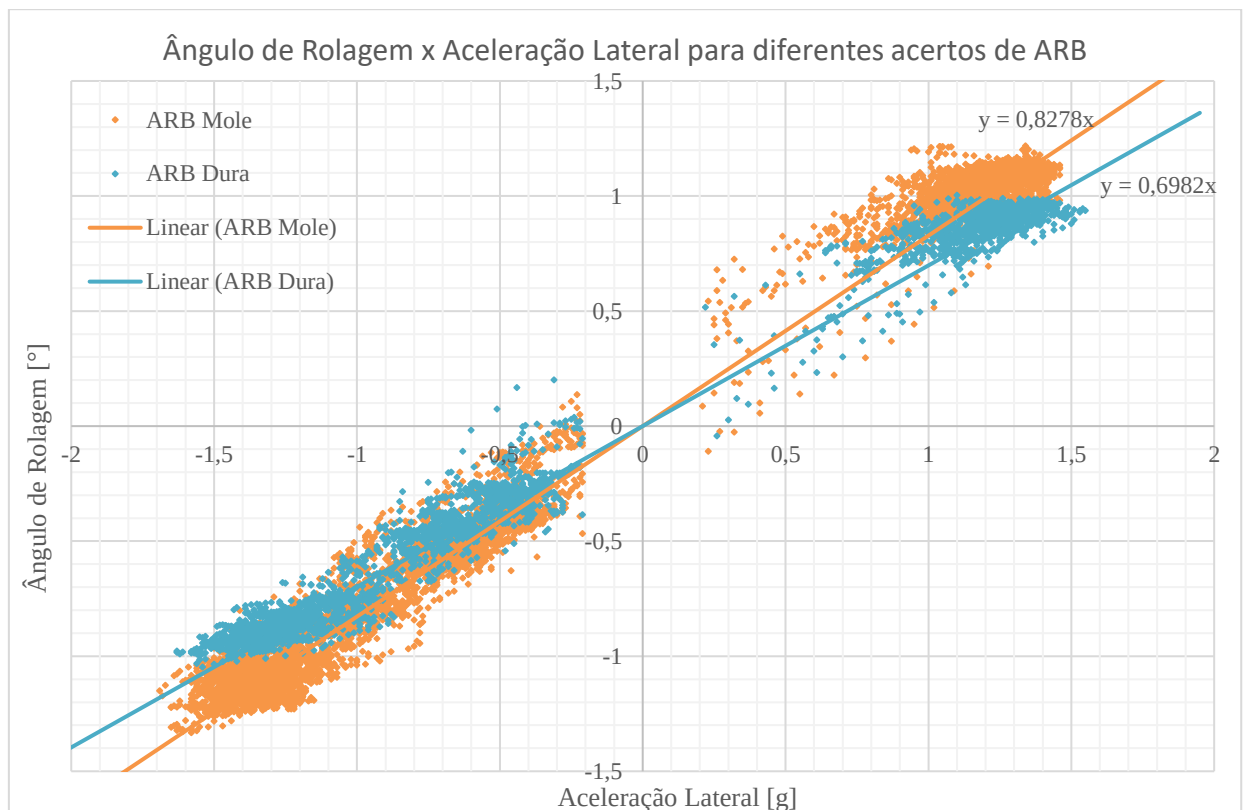
Nesse gráfico, cada ponto representa um instante na pista. É possível perceber que quanto maior a pressão de freio aplicada, mais a distribuição de freio tende para um valor fixo, próximo de 50%. Em pressões mais baixas, variações baixas na pressão acabam influenciando bastante na conta do balanço, por isso a alta variância. Porém, à medida que pressão é aplicada no pedal, essas pequenas variações começam a ser insignificantes, e o próprio sistema mecânico vai estabilizando, gerando a tendência que pode ser vista pelas linhas vermelhas adicionadas. Valores menores de variação do balanço em pressões mais altas significam um sistema mecânico mais constante e previsível. A análise desse gráfico ao longo de testes e do ano permite entender de maneira simples a estabilidade do sistema de freio. Também é possível comparar com carros de outros anos para perceber se houve ou não uma evolução.

Algumas análises necessitam de canais matemáticos mais avançados, então os dados são exportados e os gráficos são gerados em outro programa. O próximo exemplo irá mostrar o resultado de diferentes acertos na suspensão, utilizando os sensores de curso do amortecedor. Para isso, é necessário calcular o ângulo de rolagem do carro no eixo traseiro. A fórmula para isso é dada pela Equação 5, na qual 1.76 é referente ao *motion ratio* da suspensão, 1200mm é a bitola do veículo (distância entre as rodas de um mesmo eixo) e *Susp Pos Left* e *Susp Pos Right* são os potenciômetros de suspensão de um mesmo eixo:

$$Roll\ Angle[^\circ] = \text{atan} \left(('Susp\ Pos\ Left'[mm] - 'Susp\ Pos\ Rear'[mm]) * \frac{1.76}{1200} \right) \quad (5)$$

Com isso, é possível gerar os canais de rolagem dianteira e traseira do carro. Dinamicamente, o ângulo de rolagem é diretamente proporcional com a aceleração do veículo, logo é comum utilizar diagramas de dispersão de rolagem e aceleração lateral. A Figura 52 mostra um gráfico dos dados de um teste de *skidpad*.

Figura 52: Gráfico de dispersão do ângulo de rolagem por aceleração lateral



Fonte: Autor

O gráfico possui duas séries, cada uma está relacionada a um acerto diferente na suspensão. A série azul está com a barra estabilizadora (ARB) mais dura, o que diminui o movimento de rolagem do carro. A série laranja possui um acerto de ARB mais mole. É possível perceber que para um mesmo valor de aceleração lateral, a série azul possui valores menores de ângulo de rolagem, o que comprova a funcionalidade do acerto. É possível quantificar esse efeito gerando uma linha de tendência linear, e analisando o valor da tangente dessa linha. Nesse caso, esse valor é conhecido como gradiente de rolagem, e indica quantos graus de rolagem o protótipo irá sofrer para cada g de aceleração lateral. Assim, é possível analisar o gradiente de cada um dos acertos e comparar a diferença entre eles, além de poder comparar com valores de simulação para validar o projeto.

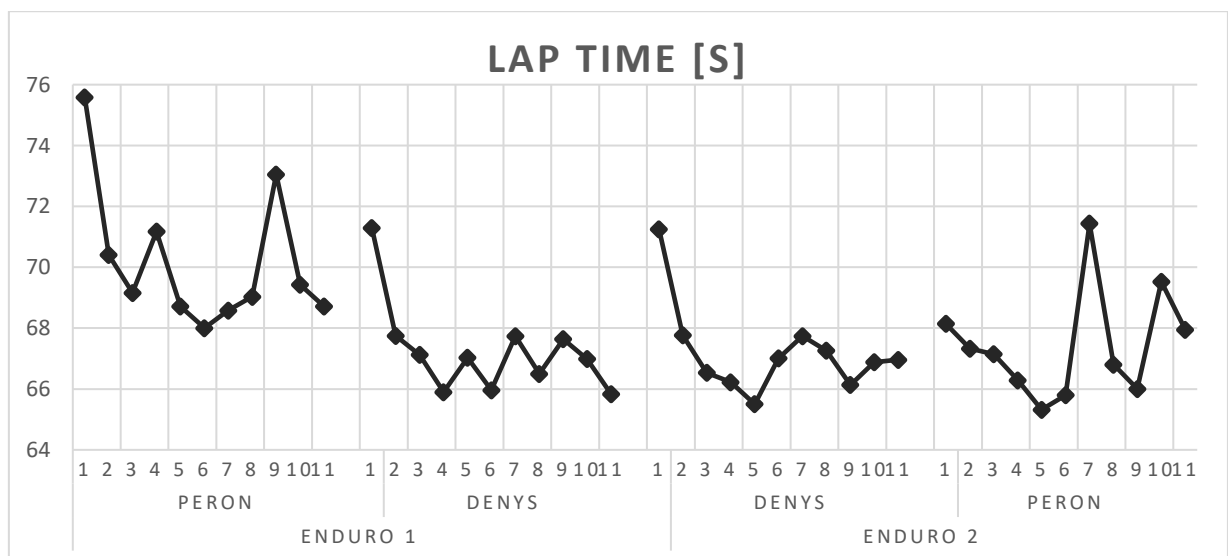
4.2.3 Indicadores de desempenho

As análises mostradas nos dois itens anteriores permitem um bom entendimento do carro e da pilotagem, porém um teste que dura um dia inteiro possui uma quantidade de dados muito grande, e é difícil saber qual dado de qual volta deve ser analisado, principalmente durante o teste em si que o tempo de análise é limitado. Para diminuir esse problema, indicadores de desempenho são utilizados. Esses indicadores têm como objetivo combinar vários canais obtidos em um único valor para tentar caracterizar parâmetros como confiança do piloto e balanço do carro. O cálculo desses indicadores sintetiza uma grande quantidade de dados em um único valor, em geral por volta, e permite identificar tendências e valores facilmente.

Para possibilitar o uso desses indicadores de maneira rápida e efetiva, foi criada uma planilha com *scripts* escrito em VBA, possibilitando automatizar as funções para gerar os gráficos, diminuindo assim drasticamente o tempo necessário para gerar os gráficos. Com isso, é possível utilizar desses indicadores em tempos curtos de análise, como no meio de um teste.

O indicador de desempenho mais importante é o próprio tempo de volta. A Figura 53 mostra um gráfico mostrando o tempo de voltas ao longo de um teste. No eixo horizontal é possível ver o número da volta, o piloto dirigindo e a sessão, enquanto no eixo vertical é mostrado o tempo de volta em segundos.

Figura 53: Progressão do tempo de volta em um teste

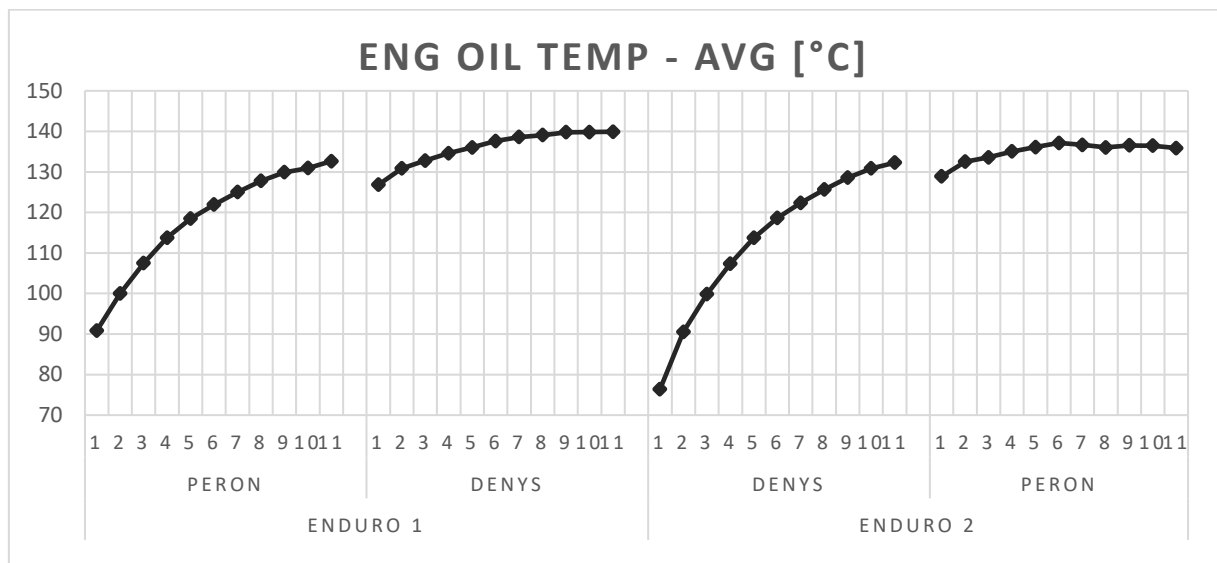


Fonte: Autor

Analisando o indicador de tempo de volta, é possível perceber a evolução dos pilotos ao longo do teste, além de identificar rapidamente voltas abaixo ou acima da média, e se for o caso, abrir os dados da volta específica para entender o que de fato aconteceu.

Esse tipo de gráfico usado mostra os dados de forma cronológica, o que é muito útil para entender a evolução do carro. A Figura 54 por exemplo mostra o mesmo gráfico com o indicador de desempenho referente a temperatura de óleo média por volta.

Figura 54: Temperatura média de óleo em um teste

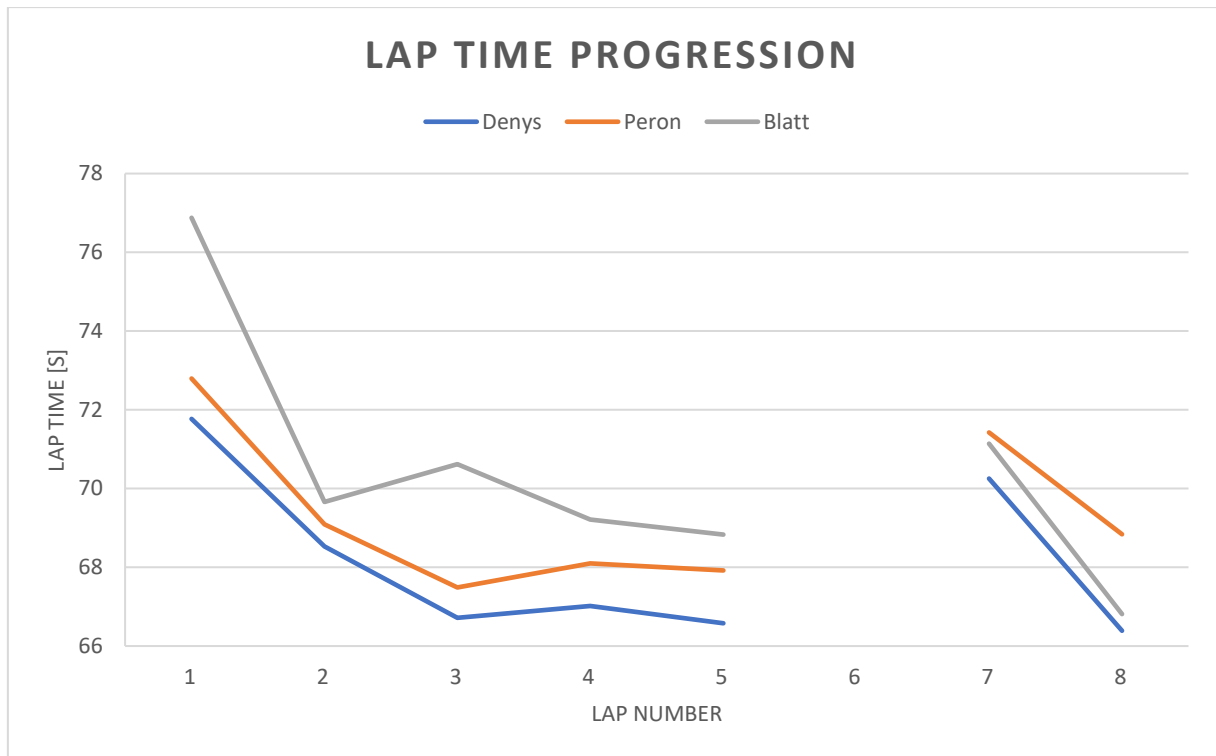


Fonte: Autor

Esse gráfico é muito importante pois mostra o comportamento da temperatura de óleo ao longo das simulações da prova de *endurance*. É muito importante que a temperatura estabilize em uma faixa ideal e não aumente demasiadamente. Na primeira simulação, é possível ver que a temperatura do óleo estava em 90 graus, e ao final da simulação estava em 140 graus. Já na segunda simulação, a temperatura começou em 75 graus e terminou em 135 graus.

Um outro tipo de gráfico utilizado é usado para foco em comparação de pilotos, em que os dados de pilotos diferentes são sobrepostos, a fim de facilitar a comparação entre eles. A Figura 55 mostra os tempos de voltas de 3 pilotos diferentes em um teste de voltas rápidas. O teste é composto de uma saída de 5 voltas para o piloto conhecer a pista, seguida por uma saída de 2 voltas afim de estabelecer sua volta mais rápida. No gráfico, o espaço vazio entre as voltas 5 e 7 representam que o piloto parou o carro e fez outra saída.

Figura 55: Comparação tempo de volta entre pilotos

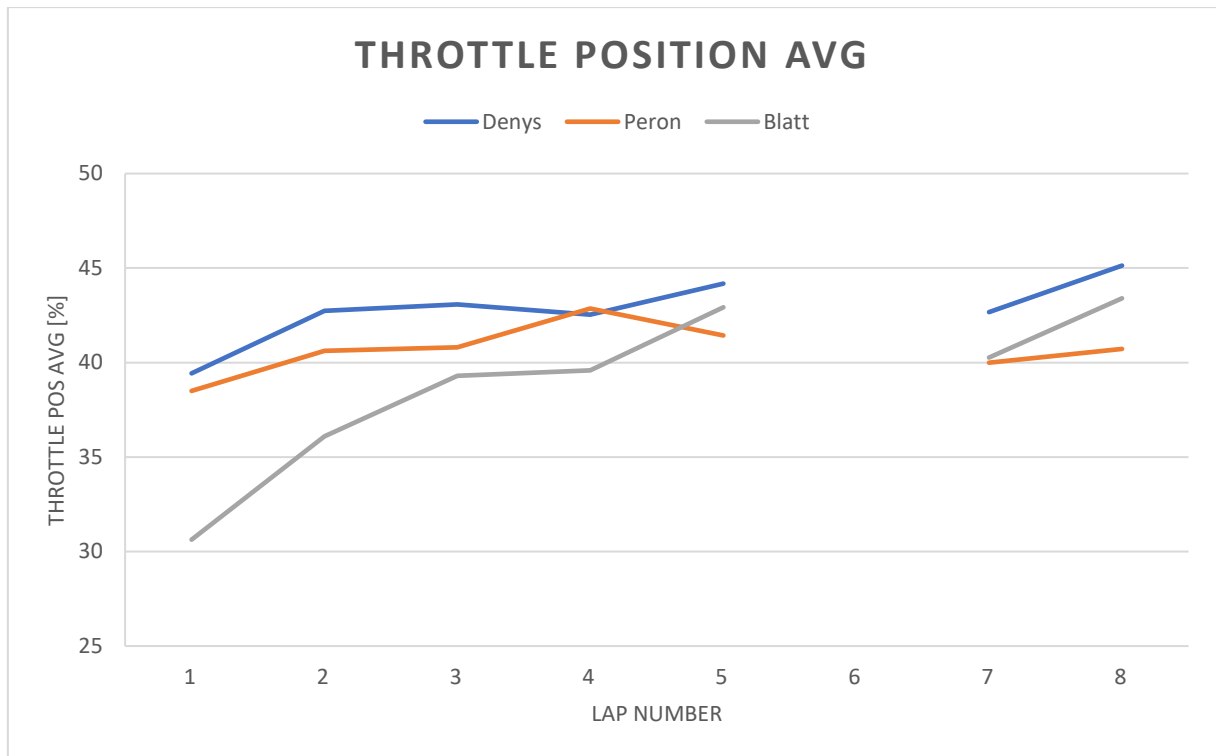


Fonte: Autor

Por esse gráfico, é possível perceber que todos os pilotos tiveram uma evolução no tempo de volta parecida, sendo que os pilotos representados pelas curvas azul e laranja tiveram uma constância maior no início, e conseguiram nas 3 últimas voltas estabilizar o tempo de volta. Porém, na última volta, o piloto laranja ficou abaixo do desempenho dos outros dois. Isso acabou acontecendo devido uma troca de marcha errada que custou o tempo do piloto.

Além do tempo de voltas, existem vários outros indicadores matemáticos com foco na análise de pilotagem. Alguns exemplos bastante usados são a posição média do pedal de acelerador, posição média do pedal de freio, e o total esterçado pelo piloto. A Figura 56 mostra um gráfico da comparação da posição média do acelerador dos três pilotos.

Figura 56: Comparação do acelerador entre pilotos

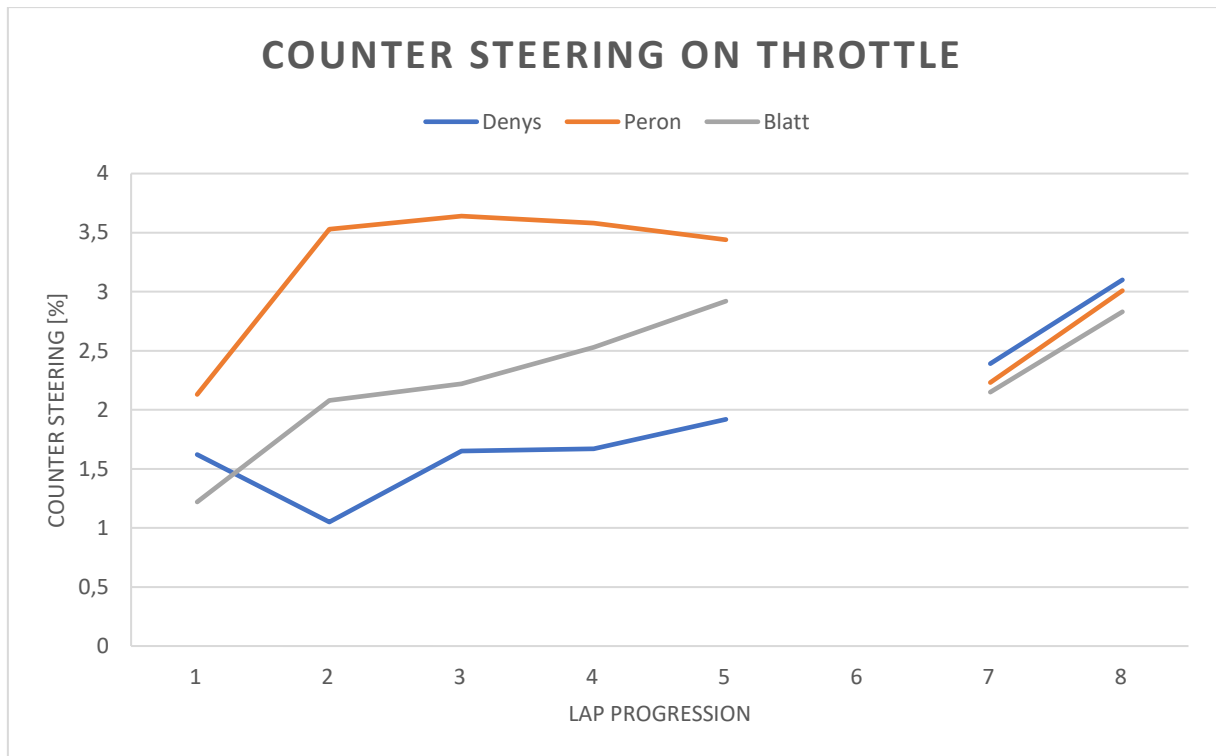


Fonte: Autor

Aqui, é possível perceber que o piloto cinza nas primeiras voltas está bem abaixo dos outros dois pilotos, o que pode ter gerado tempos de voltas menores como visto anteriormente. Ao final do teste, os três pilotos estão com a posição do pedal em um nível semelhante. O nível médio do pedal do acelerador normalmente está relacionado com a confiança do piloto no carro, e um valor mais alto é em geral desejado.

Por fim, um último exemplo mostrado é o indicador de contra esterço, que é proporcional a quantidade de tempo que o piloto gasta em estado de contra esterço. Esse estado é definido quando o sensor de esterço indica o piloto apontando o volante para o lado contrário no qual o protótipo está rotacionado (definido pelo giroscópio). Além disso, foi definido também uma condição de considerar contra esterço apenas quando o piloto está pisando no acelerador. A Figura 57 mostra o gráfico do contra esterço no acelerador dos três pilotos.

Figura 57: Contra esterço ao longo das voltas

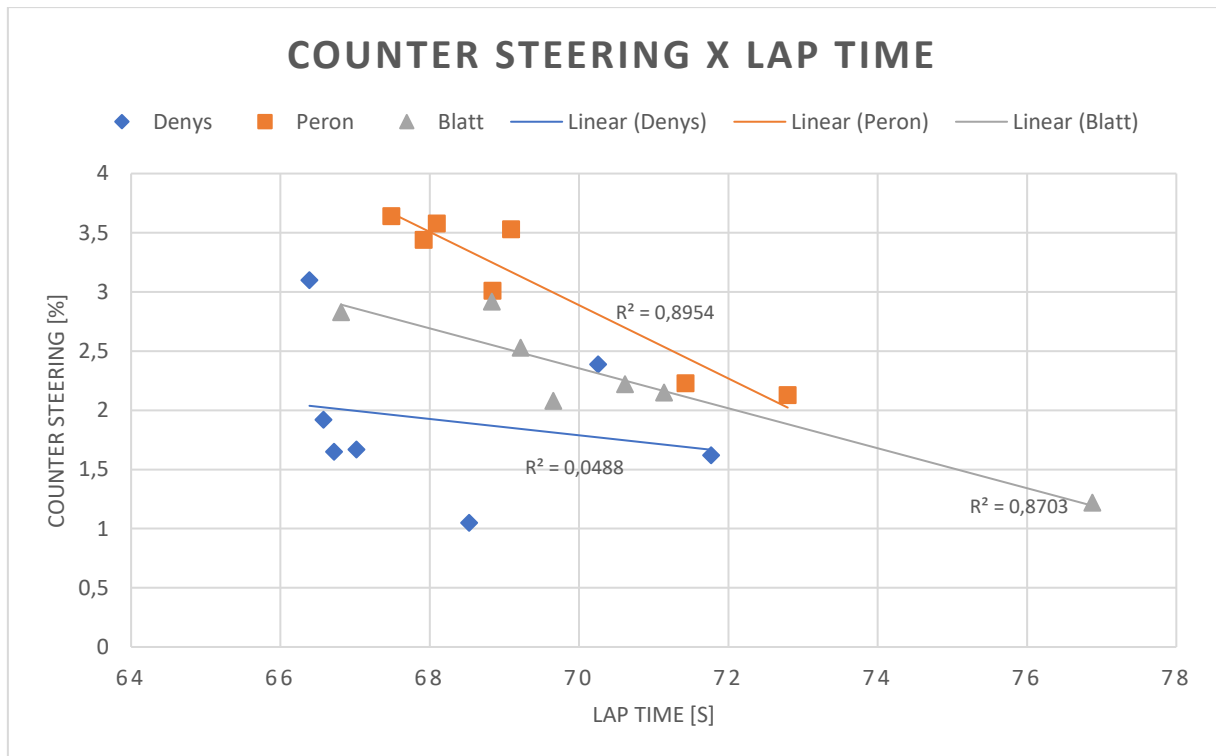


Fonte: Autor

O eixo vertical nesse caso indica a porcentagem de tempo na condição especificada. É possível perceber que os 3 pilotos possuem comportamentos diferentes entre si nas 3 primeiras. O piloto laranja é o mais agressivo, pois é o que mais necessita realizar correções nas acelerações, e mantém um valor constante ao longo das voltas. O piloto azul começa com uma porcentagem bem baixa, mas ao longo das voltas vai aumentando, assim como o piloto cinza. Porém, é interessante perceber que nas voltas duas finais, todos os pilotos tiveram um comportamento extremamente parecido.

Ao cruzar o índice de contra esterço com o tempo da volta, é possível gerar um diagrama de dispersão que permite ter uma ideia do quanto o indicador está relacionado com tempo de volta, como mostra o gráfico da Figura 58.

Figura 58: Contra esterço por tempo de volta



Fonte: Autor

É possível perceber pelas linhas de tendências, que todos os pilotos foram mais rápidos com um valor maior de contra esterço, porém o quanto isso influenciou varia de piloto para piloto. O piloto laranja mostrou uma relação forte entre contra esterço e tempo de volta, obtendo os melhores tempos com valores altos. O piloto cinza também apresentou uma relação forte entre as duas variáveis, mas com uma influência (inclinação) menor. O piloto azul por outro lado, por mais que obteve seu melhor tempo de volta com um valor alto de contra esterço, não obteve uma relação forte entre as duas variáveis, visto pelo coeficiente de determinação de 0,048. A partir dessa análise, foi possível abrir as voltas individuais de cada piloto e analisar de fato como cada piloto se comportava nessas condições, e se de fato eles estavam ganhando tempo nesses setores.

É importante notar que os indicadores de desempenho não permitem, na maioria das vezes, tirar conclusões absolutas, mas permitem perceber variáveis e fatores que poderiam ser ignorados em outras análises, além de fornecer um ponto de partida para as análises mais detalhadas.

5 CONCLUSÃO

Em ambientes de competições automotivas, é de extrema importância garantir a confiabilidade dos sistemas críticos e extrair o máximo de desempenho possível do carro, e a implementação do sistema elétrico e eletrônico do carro é de extrema importância para alcançar esses objetivos. Sistemas comerciais existem e são bastante utilizados por equipes profissionais, porém o altíssimo custo desses materiais impossibilita o uso por uma equipe de Fórmula SAE. Assim, um sistema completo de aquisição de dados e distribuição de energia foi desenvolvido que cumprisse com os requisitos necessários pela equipe.

O módulo de distribuição de energia permitiu remover os relés eletromecânicos do carro, reduzindo o espaço necessário, e não apresentou nenhum problema durante os períodos de teste e da competição após o projeto ter sido finalizado e testado inicialmente.

O uso de atuadores permitiu um maior desempenho do carro e dos pilotos. O sistema de troca de marchas com uso de borboletas reduz o tempo necessário para as trocas e diminui consideravelmente a fadiga dos pilotos. O sistema de embreagem acionada por um botão permite a configuração do perfil de largada, e junto com um sistema de troca de marcha automático, permite a padronização da largada para a prova de aceleração, reduzindo ao máximo possível a interferência humana nos tempos. O uso do DRS permite também uma maior aceleração durante as provas de *autocross* e enduro.

Alguns testes feitos chegaram à conclusão de que o ganho do sistema de troca de marchas é de aproximadamente 1s por volta, com um tempo base de 1:10s, além de uma melhora de 0,3s na prova de aceleração, que costuma ter tempos de 4,2s. Já o uso do DRS gerou um tempo aproximadamente 0,6s mais rápido por volta, em uma volta de 1:07s.

O sistema de aquisição de dados permitiu melhorar a pilotagem de todos os pilotos, com um ganho maior para os pilotos novos, que conseguem ver claramente os trechos onde estão perdendo tempo, e assim conseguem melhorar o estilo de pilotagem. Além disso, os subsistemas do carro utilizam da aquisição para validar o projeto, testar modificações e modificar o projeto no ano seguinte. Um exemplo, utilizando os sensores de curso de suspensão, foi analisado que o carro aumentava muito sua altura durante curvas, e assim a geometria da suspensão do ano seguinte levou isso em consideração, e comparando os dados dos dois anos foi possível verificar que as mudanças de fato fizeram efeito.

Assim, é possível concluir que os principais objetivos iniciais foram cumpridos, e o sistema funcionou da maneira prevista, garantindo a confiabilidade e aumentando o

desempenho do veículo. Além disso, o sistema elétrico foi o mais bem avaliado da competição de FSAE Brasil de 2019, na categoria combustão, com uma nota de 24,6/25.

5.1 Trabalhos Futuros

O projeto desenvolvido ainda possui bastantes oportunidades de melhorias, que necessitam de mais tempo para serem desenvolvidas, mas permitem um maior sensoramento do carro.

Inicialmente se recomenda o uso de circuitos integrados automotivos no módulo de distribuição de energia, que permitem o sensoramento de corrente e funções que simulem um fusível.

Uma outra recomendação é a implementação de um sistema de malha fechada nas trocas de marcha, que necessita da instalação de um sensor analógico da posição atual do câmbio. Um controle de malha fechada irá diminuir as trocas que podem falhar e irá desgastar menos o sistema como um todo.

Por fim, o uso de outros sensores irá aumentar o entendimento da dinâmica do carro. O principal sensor a ser implementado é a célula de carga na suspensão, que permite ter um entendimento das cargas que passam pelo sistema. Sensores de temperatura infravermelho também são bastante úteis para medir componentes cujo desempenho depende da temperatura, como os pneus e os discos de freio.

REFERÊNCIAS

ALFERINK, Timon. **Practical tips: CAN-Bus**. Disponível em:

<<https://www.kmpdrivetrain.com/paddleshift/practical-tips-can-bus/>>. Acesso em 14 out 2020.

Analog Devices. **Three-Axis Accelerometer Evaluation Board**. Disponível em:

<<https://www.analog.com/media/en/technical-documentation/evaluation-documentation/EVAL-ADXL325Z.pdf>>. Acesso em 14 out 2020.

Analog Devices. **Vibration Rejecting $\pm 250^\circ/\text{s}$ Yaw Rate Gyroscope**. Disponível em:

<<https://www.analog.com/media/en/technical-documentation/data-sheets/ADXRS642.pdf>>. Acesso em 14 out 2020.

BARBOSA PORTO, Danilo. **Sistema ativo de redução de arrasto aerodinâmico por atuador aplicado a um protótipo de Fórmula SAE**. 2016. Trabalho de Conclusão de Curso (Graduação) – Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2016.

Components 101. **What is Optocoupler and How it works**. Disponível em:

<<https://components101.com/articles/what-is-optocoupler-and-how-it-works>>. Acesso em 14 out 2020.

CSS Eletronics. **CAN Bus Explained - A Simple Intro**. Disponível em:

<<https://www.csselectronics.com/screen/page/simple-intro-to-can-bus/language/en>>. Acesso em 14 out 2020.

Digi. **Digi XBee-PRO 900HP RF Module**. Disponível em:

<<https://www.digi.com/products/embedded-systems/digi-xbee/rf-modules/sub-1-ghz-rf-modules/xbee-pro-900hp>> Acesso em 14 out 2020.

FOWLER, Daniel. **CAN Bus Wiring Diagram, a Basics Tutorial**. Tekeye. Disponível em:

<<https://tekeye.uk/automotive/can-bus-cable-wiring>>. Acesso em 14 out 2020.

Harold G. Schaevitz Industries. **LPPS-22 Linear Potentiometer Position Sensor With Rod End Joints**. Disponível em: <<https://www.hgsind.com/product/lpps-22-linear-potentiometer-position-sensor-rod-end-joints>>. Acesso em 14 out 2020.

MEHL, Ewaldo Luiz De Mattos. **Conceitos fundamentais sobre placas de circuito impresso**. Elétrica UFPR. Disponível em:

<http://www.eletrica.ufpr.br/mehl/te232/textos/PCI_Conceitos_fundamentais.pdf>. Acesso em 14 out 2020.

SAE Brasil. **Fórmula SAE**. Disponível em: <<https://saebrasil.org.br/programas-estudantis/formula-sae-brasil/>>. Acesso em 01 out 2020.

SATTEL, Sam. **What Are Decoupling Capacitors?** Autodesk. Disponível em:

<<https://www.autodesk.com/products/eagle/blog/what-are-decoupling-capacitors/>> Acesso em 14 out 2020.

Sensata. **3PP8 Hermetic Pressure Sensors**. Disponível em:
<<https://www.sensata.com/products/pressure-sensors-switches/3pp8-hermetic-pressure-sensors>>. Acesso em 14 out 2020.

STMicroeletronics. **Positive voltage regulator ICs**. Disponível em:
<<https://www.st.com/resource/en/datasheet/l78.pdf>>. Acesso em 14 out 2020.

STMicroeletronics. **Description of STM32F4 HAL and low-layer drivers**. Disponível em:
<https://www.st.com/resource/en/user_manual/dm00105879-description-of-stm32f4-hal-and-ll-drivers-stmicroelectronics.pdf>. Acesso em 14 out 2020.

Texas Instruments. **Principles of Data Acquisition and Conversion**. Disponível em:
<<https://www.ti.com/lit/an/sbaa051a/sbaa051a.pdf>>. Acesso em 14 out 2020.

u-blox. **NEO-6 series**. Disponível em: <<https://www.u-blox.com/en/product/neo-6-series>>. Acesso em 14 out 2020.

UNICAMP. **Transistores de Efeito de Campo (FETS)**. Disponível em:
<<http://www.dsif.fee.unicamp.br/~elnatan/ee640/aula%20mosfet.pdf>>. Acesso em 14 out 2020.

University of Surrey. **Active Low-Pass Filter**. Disponível em:
<<http://www.ee.surrey.ac.uk/Projects/CAL/frequency-response/activelp.html>>. Acesso em 14 out 2020.